

전자전기컴퓨터설계실험Ⅱ

FPGA. 디지털 회로 실험 교재

목차 1/7 | 조합회로

1. 조합회로

실습 1 / AND, OR, XOR 게이트 구현	14
실습 2 / NOT 게이트 구현	19
실습 3 / 전가산기 구현	23
실습 4 / 전감산기 구현	29
실습 5 / 8X3 ENCODER 설계	34
실습 6 / 3X8 DECODER 설계	40
실습 7 / 4BIT MULTIPLEXER 설계	53
실습 8 / 1X8 DEMULTIPLEXER 설계	58
실습 9 / 크기 비교기	63
실습 13 / 4비트 덧셈 회로	67
실습 14 / 4비트 뺄셈 회로	71
실습 15 / 4비트 곱셈 회로	75
보충 / 계층적 가산기와 전수 검사	79



목차 2/7 | 순차회로

2. 순차회로

첫 순차회로: 카운터	81
실습 16 / 카운터 회로 설계	86
실습 17 / 클럭 분주 회로 설계	90
실습 18 / 스텝 모터 제어 회로 설계	97
실습 26 / 데이터의 저장과 이동	102
실습 27 / 시프트 레지스터 구현	108
실습 28 / PISO 제어	113



목차 3/7 | 표시 장치

3. 표시 장치

실습 10 / BCD-2진 코드 변환기	119
실습 11 / 7-SEGMENT DECODER 설계	124
실습 12 / 7-SEGMENT ARRAY 제어	130
실습 32 / TEXT LCD 제어	135

목차 4/7 | 파형 생성과 구동

4. 파형 생성과 구동

실습 19 / PIEZO 제어	145
실습 20 / PIEZO로 피아노 만들기	151
실습 21 / 시계 만들기	159
실습 22 / PWM을 이용한 LED 밝기 제어	167
실습 23 / FULL COLOR LED 순차 제어	172
실습 24 / FULL COLOR LED PWM 제어	177



목차 5/7 | 상태 머신

5. 상태 머신

실습 29 / 무어 스테이트 머신 설계	190
실습 30 / 밀리 스테이트 머신 구현	195
응용 / 상태 머신 통합 제어	200

목차 6/7 | 인터페이스와 통합 검증

6. 인터페이스와 통합 검증

UART 8N1	209
통합 시뮬레이션	214
실습별 VCD 시간 구간	216
통합 계층과 보충 모듈 코드	219
단일 테스트벤치 전체 코드	270



목차 7/7 | 출처와 보충 자료

7. 출처와 보충 자료

교육 자료 출처
장비 제공 보충 소스

325

326

교재와 프로젝트의 범위

- 2022년 교안의 30개 번호 실습, 첫 카운터, 상태 머신 응용을 주제별로 재작성했다.
- 프로젝트 생성과 합성 절차의 반복 화면은 제외했다.
- 새 코드는 37개 설계 모듈과 테스트벤치 하나다. 6개 계층에서 33개 실습·보충 항목을 검증한다.
- 구형 보드 핀맵과 시뮬레이션용 입출력을 구분한다. 보드 연결 전 XDC를 대조한다.



하나의 프로젝트, 여섯 개 계층

- tb_legacy / dut / u_comb: 게이트·선택·산술
- u_seq: 카운터·저장·시프트
- u_display: BCD·7세그먼트·LCD 명령
- u_waveform: PWM·음원·모터 위상
- u_fsm: Moore·Mealy·상태 전이
- u_interface: UART 송수신 루프백



분기와 반복을 회로로 표현하기

- 조합회로의 if/case는 입력에 따라 결과를 선택하는 회로를 표현한다.
- 클록마다 상태를 갱신하면 시간에 따른 반복 동작을 만든다.
- 합성 가능한 고정 횟수 for는 반복되는 하드웨어를 기술할 수 있다.
- 테스트벤치의 반복문은 입력을 바꾸며 회로를 검사한다. 같은 문법도 역할이 다르다.

원자료를 읽을 때의 주의점

- 원자료의 LVCM0SS33 표기는 오타다. XDC의 실제 속성값은 LVCM0S33이다.
- 표시된 핀 번호는 그 페이지의 원본 예제에 해당한다. 새 탑모듈의 포트에 그대로 복사하지 않는다.
- 클록을 버튼으로 직접 공급하는 과거 예제와 단일 기준 클록으로 동작하는 새 코드를 구분한다.
- 원본과 새 코드의 비트 순서·폭·상태 동작이 다르면 각 실습에서 차이를 밝힌다.



조합회로 / 구성

1. 조합회로

실습 1 / AND, OR, XOR 게이트 구현	14
실습 2 / NOT 게이트 구현	19
실습 3 / 전가산기 구현	23
실습 4 / 전감산기 구현	29
실습 5 / 8X3 ENCODER 설계	34
실습 6 / 3X8 DECODER 설계	40
실습 7 / 4BIT MULTIPLEXER 설계	53
실습 8 / 1X8 DEMULTIPLEXER 설계	58
실습 9 / 크기 비교기	63
실습 13 / 4비트 덧셈 회로	67
실습 14 / 4비트 뺄셈 회로	71
실습 15 / 4비트 곱셈 회로	75
보충 / 계층적 가산기와 전수 검사	79



실습 1 / AND, OR, XOR 게이트 구현

a	b	AND	OR	XOR
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

- AND는 모두 1, OR는 하나 이상 1, XOR는 입력이 다를 때 1이다.
- 비트 연산자는 &, |, ^다. 논리 연산자와 구분한다.
- 입력 4가지 조합을 모두 검사한다. 조합회로이므로 클록은 필요하지 않다.

실습 1 / 핀 참고

원래 예제의 포트 이름과 패키지 핀. 새 시뮬레이션 탭에는 적용하지 않는다.

포트	핀	포트	핀
a	K4	y	M4
x	L4	z	M2
b	N8		



실습 1 / AND, OR, XOR 게이트 / 시뮬레이션 확인

확인할 계층

tb_legacy.dut.u_comb.u_gates

- a,b에 00, 01, 10, 11을 넣는다.
- and_out, or_out, xor_out을 진리표의 같은 행과 비교한다.
- 입력을 바꾸기 전에 예상 출력을 적고, VCD의 값과 비교한다.



실습 1 / AND, OR, XOR 게이트 / 코드

src/combinational/logic_gates.v

L1-12

```
1 `timescale 1ns/1ps
2 `default_nettype none
3
4 // 실습 1~2: 비트 단위 논리 연산.
5 module logic_gates (
6     input wire a,
7     input wire b,
8     output wire and_out,
9     output wire or_out,
10    output wire xor_out,
11    output wire not_out
12 );
```



실습 1 / AND, OR, XOR 게이트 / 코드

src/combinational/logic_gates.v

L13-19

```
13     assign and_out = a & b;
14     assign or_out = a | b;
15     assign xor_out = a ^ b;
16     assign not_out = ~a;
17 endmodule
18
19 `default_nettype wire
```



실습 2 / NOT 게이트 구현

a	z
0	1
1	0

- 비트 반전은 $\sim a$. 0과 1을 서로 바꾼다.
- 버스 반전은 모든 비트에 적용된다. 논리 부정 !는 입력 전체가 0인지 판정한다.
- 입력이 바뀌면 조합 논리를 거쳐 출력에 반영된다.



실습 2 / 핀 참고

원래 예제의 포트 이름과 패키지 핀. 새 시뮬레이션 탭에는 적용하지 않는다.

포트	핀	포트	핀
a	K4	z	L4



실습 2 / NOT 게이트 / 시뮬레이션 확인

확인할 계층

tb_legacy.dut.u_comb.u_gates

- a에 0과 1을 번갈아 넣는다.
- not_out은 각각 1과 0이다. 클록을 기다리는 회로가 아니다.
- 입력을 바꾸기 전에 예상 출력을 적고, VCD의 값과 비교한다.



실습 2 / NOT 게이트 / 코드 연결

src/combinational/logic_gates.v

전체 코드는 p. 17부터 수록했다.

이 실습의 입력·출력은 앞의 확인 항목과 함께 읽는다.



실습 3 / 전가산기 구현

a	b	cin	sum	cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

- $\text{sum} = a \oplus b \oplus \text{cin}$. 입력 세 비트의 합을 두 비트로 표현한다.
- $\text{cout} = ab + (a \oplus b)\text{cin}$. 반가산기 두 개와 OR로 구성한다.
- 새 full_adder는 half_adder 두 개를 연결한 계층 구조다.

실습 3 / 핀 참고

원래 예제의 포트 이름과 패키지 핀. 새 시뮬레이션 탑에는 적용하지 않는다.

포트	핀	포트	핀
a	K4	cout	M4
s	L4	cin	N4
b	N8		



실습 3 / 전가산기 / 시뮬레이션 확인

확인할 계층

tb_legacy.dut.u_comb.u_bit1

- a,b,carry_in의 8가지 조합을 모두 검사한다.
- {carry_out,sum}은 세 입력의 합이다. 1+1+1의 결과는 11이다.
- 입력을 바꾸기 전에 예상 출력을 적고, VCD의 값과 비교한다.



실습 3 / 전가산기 / 코드

src/combinational/full_adder.v

L1-12

```
1  `timescale 1ns/1ps
2  `default_nettype none
3
4  // 전가산기: 반가산기 두 개를 연결한다.
5  module full_adder (
6      input wire a,
7      input wire b,
8      input wire carry_in,
9      output wire sum,
10     output wire carry_out
11 );
12     wire first_sum, first_carry, second_carry;
```



실습 3 / 전가산기 / 코드

src/combinational/full_adder.v

L13-24

```
13
14     half_adder u_first (
15         .a(a),
16         .b(b),
17         .sum(first_sum),
18         .carry(first_carry)
19     );
20
21     half_adder u_second (
22         .a(first_sum),
23         .b(carry_in),
24         .sum(sum),
```



실습 3 / 전가산기 / 코드

src/combinational/full_adder.v

L25-31

```
25         .carry(second_carry)
26     );
27
28     assign carry_out = first_carry | second_carry;
29 endmodule
30
31 `default_nettype wire
```



실습 4 / 전감산기 구현

a	b	bin	diff	borrow
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

- $\text{diff} = a \oplus b \oplus \text{bin}$. 자리빌림 입력까지 빼는 회로다.
- $\text{borrow} = (\bar{a}b) + (\overline{a \oplus b} \text{bin})$.
- 0-1-1=-2는 $\text{diff}=0$, $\text{borrow}=1$ 이다. 원자료 c/cin은 bin으로 통일한다.

실습 4 / 핀 참고

원래 예제의 포트 이름과 패키지 핀. 새 시뮬레이션 탑에는 적용하지 않는다.

포트	핀	포트	핀
a	K4	bor	M4
d	L4	cin	N4
b	N8		



실습 4 / 전감산기 / 시뮬레이션 확인

확인할 계층

tb_legacy.dut.u_comb.u_subtractor

- a,b,borrow_in의 8가지 조합을 넣는다.
- 0-1-1에서 difference=0, borrow_out=1인지 확인한다.
- 입력을 바꾸기 전에 예상 출력을 적고, VCD의 값과 비교한다.



실습 4 / 전감산기 / 코드

src/combinational/full_subtractor.v

L1-12

```
1  `timescale 1ns/1ps
2  `default_nettype none
3
4  // 실습 4: 자리빌림 입력을 포함한 1비트 뺄셈.
5  module full_subtractor (
6      input wire a,
7      input wire b,
8      input wire borrow_in,
9      output wire difference,
10     output wire borrow_out
11 );
12     assign difference = a ^ b ^ borrow_in;
```



실습 4 / 전감산기 / 코드

src/combinational/full_subtractor.v

L13-16

```
13      assign borrow_out = (~a & b) | (~(a ^ b) & borrow_in);  
14  endmodule  
15  
16  `default_nettype wire
```



실습 5 / 8X3 ENCODER 설계

i[7:0]	a[2:0]
10000000	000
01000000	001
00100000	010
00010000	011
00001000	100
00000100	101
00000010	110
00000001	111

- 원자료는 MSB 쪽 입력을 0번으로 부호화한다. 비트 번호를 그대로 출력하는 방식과 다르다.
- 한 비트만 1인 one-hot 입력을 전제로 한다.
- 입력이 0이거나 여러 비트가 1이면 valid=0이다. 우선순위 인코더와 구분한다.

실습 5 / 핀 참고

원래 예제의 포트 이름과 패키지 핀. 새 시뮬레이션 탑에는 적용하지 않는다.

포트	핀	포트	핀
i[7]	K4	a[2]	L4
i[1]	L5	i[3]	P6
i[6]	N8	a[1]	M4
i[0]	J2	i[2]	N6
i[5]	N4	a[0]	M2
i[4]	N1		



실습 5 / 8X3 ENCODER / 시뮬레이션 확인

확인할 계층

tb_legacy.dut.u_comb.u_encoder

- one_hot=10000000이면 code=0, 00000001이면 code=7이다.
- 모두 0 또는 여러 비트가 1이면 valid=0이다. code만 보고 유효성을 판단하지 않는다.
- 입력을 바꾸기 전에 예상 출력을 적고, VCD의 값과 비교한다.



실습 5 / 8X3 ENCODER / 코드

src/combinational/encoder8to3.v

L1-12

```
1  `timescale 1ns/1ps
2  `default_nettype none
3
4  // 실습 5: 원교안의 MSB-first 순서. 잘못된 입력은 valid=0.
5  module encoder8to3 (
6      input wire [7:0] one_hot,
7      output reg [2:0] code,
8      output reg valid
9  );
10     always @* begin
11         valid = 1'b1;
12         case (one_hot)
```



실습 5 / 8X3 ENCODER / 코드

src/combinational/encoder8to3.v

L13-24

```
13      8'b10000000: code = 3'd0;
14      8'b01000000: code = 3'd1;
15      8'b00100000: code = 3'd2;
16      8'b00010000: code = 3'd3;
17      8'b00001000: code = 3'd4;
18      8'b00000100: code = 3'd5;
19      8'b00000010: code = 3'd6;
20      8'b00000001: code = 3'd7;
21      default: begin
22          code = 3'd0;
23          valid = 1'b0;
24      end
```



실습 5 / 8X3 ENCODER / 코드

src/combinational/encoder8to3.v

L25-29

```
25         endcase
26     end
27 endmodule
28
29 `default_nettype wire
```



실습 6 / 3X8 DECODER 설계

abc	o[7:0]
000	00000001
001	00000010
010	00000100
011	00001000
100	00010000
101	00100000
110	01000000
111	10000000

- 3비트 입력으로 8개 출력 중 하나를 선택한다. 000은 o[0]을 켜다.
- 새 코드: `8'b0000_0001 << select`. 결과 폭은 8비트다.
- 앞 인코더의 MSB-first 규칙과 반대이므로 단순 직결하면 역함수가 되지 않는다.



실습 6 / 핀 참고

원래 예제의 포트 이름과 패키지 핀. 새 시뮬레이션 탑에는 적용하지 않는다.

포트	핀	포트	핀
a	K4	o[2]	M3
o[5]	M2	o[7]	L4
b	N8	o[1]	M1
o[4]	N7	o[6]	M4
c	N4	o[0]	N5
o[3]	M7		



실습 6 / 3X8 DECODER / 시뮬레이션 확인

확인할 계층

tb_legacy.dut.u_comb

- select를 0부터 7까지 바꾼다.
- decoded는 00000001부터 10000000까지 이동한다. 항상 한 비트만 1이다.
- 입력을 바꾸기 전에 예상 출력을 적고, VCD의 값과 비교한다.



실습 6 / 3X8 DECODER / 코드

src/combinational/lab_combinational.v

L1-12

```
1  `timescale 1ns/1ps
2  `default_nettype none
3
4  // 조합회로 묶음: 각 중간 신호는 파형 창에서 이름으로 확인한다.
5  module lab_combinational (
6      input wire [7:0] a,
7      input wire [7:0] b,
8      input wire [2:0] select,
9      output wire [31:0] result
10 );
11     wire [4:0] sum;
12     wire [4:0] difference;
```



실습 6 / 3X8 DECODER / 코드

src/combinational/lab_combinational.v

L13-24

```
13    wire [7:0] product;
14    wire [7:0] decoded;
15    wire selected_bit;
16    wire [3:0] carry;
17    wire gate_and, gate_or, gate_xor, gate_not;
18    wire subtract_bit, borrow;
19    wire [2:0] encoded;
20    wire encoder_valid, mux4;
21    wire [7:0] demux8;
22
23    // 전가산기 4개의 carry를 차례대로 연결한다.
24    full_adder u_bit0 (
```



실습 6 / 3X8 DECODER / 코드

src/combinational/lab_combinational.v

L25-31

```
25      .a(a[0]),  
26      .b(b[0]),  
27      .carry_in(1'b0),  
28      .sum(sum[0]),  
29      .carry_out(carry[0])  
30  );  
31
```



실습 6 / 3X8 DECODER / 코드

src/combinational/lab_combinational.v

L32-43

```
32     full_adder u_bit1 (  
33         .a(a[1]),  
34         .b(b[1]),  
35         .carry_in(carry[0]),  
36         .sum(sum[1]),  
37         .carry_out(carry[1])  
38     );  
39  
40     full_adder u_bit2 (  
41         .a(a[2]),  
42         .b(b[2]),  
43         .carry_in(carry[1]),
```



실습 6 / 3X8 DECODER / 코드

src/combinational/lab_combinational.v

L44-55

```
44         .sum(sum[2]),
45         .carry_out(carry[2])
46     );
47
48     full_adder u_bit3 (
49         .a(a[3]),
50         .b(b[3]),
51         .carry_in(carry[2]),
52         .sum(sum[3]),
53         .carry_out(carry[3])
54     );
55
```



실습 6 / 3X8 DECODER / 코드

src/combinational/lab_combinational.v

L56-61

```
56     assign sum[4] = carry[3];
57     assign difference = {1'b0, a[3:0]} - {1'b0, b[3:0]};
58     assign product = a[3:0] * b[3:0];
59     assign decoded = 8'b0000_0001 << select;
60     assign selected_bit = a[selected];
61
```



실습 6 / 3X8 DECODER / 코드

src/combinational/lab_combinational.v

L62-73

```
62    // 출력 비트 배치는 README 참고. 세부 신호도 VCD에 기록된다.
63    assign result = {a[0] & b[0], a[0] | b[0], a[0] ^ b[0],
64                    a[3:0] == b[3:0], a[3:0] > b[3:0], selected_bit,
65                    decoded, product, difference, sum};
66
67    logic_gates u_gates (
68        .a(a[0]),
69        .b(b[0]),
70        .and_out(gate_and),
71        .or_out(gate_or),
72        .xor_out(gate_xor),
73        .not_out(gate_not)
```



실습 6 / 3X8 DECODER / 코드

src/combinational/lab_combinational.v

L74-85

```
74     );  
75  
76     full_subtractor u_subtractor (  
77         .a(a[0]),  
78         .b(b[0]),  
79         .borrow_in(select[0]),  
80         .difference(subtract_bit),  
81         .borrow_out(borrow)  
82     );  
83  
84     encoder8to3 u_encoder (  
85         .one_hot(a),
```



실습 6 / 3X8 DECODER / 코드

src/combinational/lab_combinational.v

L86-95

```
86         .code(encoded),
87         .valid(encoder_valid)
88     );
89
90     mux4to1 u_mux4 (
91         .data(a[3:0]),
92         .select(select[1:0]),
93         .selected(mux4)
94     );
95
```



실습 6 / 3X8 DECODER / 코드

src/combinational/lab_combinational.v

L96-104

```
96     demux1to8 u_demux8 (  
97         .data(b[0]),  
98         .select(select),  
99         .routed(demux8)  
100     );  
101  
102 endmodule  
103  
104 `default_nettype wire
```



실습 7 / 4BIT MULTIPLEXER 설계

sel[1:0]	z
00	i[3]
01	i[2]
10	i[1]
11	i[0]

- 여러 입력 중 하나를 출력으로 선택한다.
- 원자료 세 번째 행의 00은 10으로 바로잡았다. MSB-first 순서는 유지한다.
- mux4to1은 표의 MSB-first 4:1 선택을 구현한다. 별도 a[select]는 보충 8:1 예제다.

실습 7 / 핀 참고

원래 예제의 포트 이름과 패키지 핀. 새 시뮬레이션 탭에는 적용하지 않는다.

포트	핀	포트	핀
i[3]	K4	i[1]	N4
s[1]	Y1	i[0]	Y7
i[2]	N8	z	L4
s[0]	W3		



실습 7 / 4BIT MULTIPLEXER / 시뮬레이션 확인

확인할 계층

tb_legacy.dut.u_comb.u_mux4

- data=1010에서 select를 00,01,10,11로 바꾼다.
- selected는 1,0,1,0이다. select=00이 data[3]을 고른다.
- 입력을 바꾸기 전에 예상 출력을 적고, VCD의 값과 비교한다.



실습 7 / 4BIT MULTIPLEXER / 코드

src/combinational/mux4to1.v

L1-12

```
1 `timescale 1ns/1ps
2 `default_nettype none
3
4 // 실습 7: sel=00이면 data[3], sel=30이면 data[0].
5 module mux4to1 (
6     input wire [3:0] data,
7     input wire [1:0] select,
8     output reg selected
9 );
10     always @* begin
11         case (select)
12             2'd0: selected = data[3];
```



실습 7 / 4BIT MULTIPLEXER / 코드

src/combinational/mux4to1.v

L13-21

```
13             2'd1: selected = data[2];
14             2'd2: selected = data[1];
15             2'd3: selected = data[0];
16             default: selected = 1'b0;
17         endcase
18     end
19 endmodule
20
21 `default_nettype wire
```



실습 8 / 1X8 DEMULTIPLEXER 설계

sel	입력을 전달할 출력
000	o[7]
001	o[6]
010	o[5]
011	o[4]
100	o[3]
101	o[2]
110	o[1]
111	o[0]

- 입력 i 를 선택된 출력으로 보내고 나머지는 0으로 둔다.
- 원자료는 000일 때 o[7], 111일 때 o[0]을 선택한다.
- 디코더 출력과 i 를 AND한 구조로 해석할 수 있다. $i=0$ 이면 모든 출력이 0이다.

실습 8 / 핀 참고

원래 예제의 포트 이름과 패키지 핀. 새 시뮬레이션 탭에는 적용하지 않는다.

포트	핀	포트	핀
i	K4	o[3]	M7
o[6]	M4	s[0]	U2
o[5]	M2	o[2]	M3
s[2]	Y1	o[1]	M1
o[4]	N7	o[7]	L4
s[1]	W3	o[0]	N5



실습 8 / 1X8 DEMULTIPLEXER / 시뮬레이션 확인

확인할 계층

tb_legacy.dut.u_comb.u_demux8

- data=1에서 select를 0부터 7까지 바꾼다.
- routed의 10이 MSB에서 LSB로 이동한다. data=0이면 전체 출력이 0이다.
- 입력을 바꾸기 전에 예상 출력을 적고, VCD의 값과 비교한다.



실습 8 / 1X8 DEMULTIPLEXER / 코드

src/combinational/demux1to8.v

L1-12

```
1  `timescale 1ns/1ps
2  `default_nettype none
3
4  // 실습 8: sel=0이면 routed[7]. 선택되지 않은 출력은 0.
5  module demux1to8 (
6      input wire data,
7      input wire [2:0] select,
8      output wire [7:0] routed
9  );
10     assign routed = data ? (8'b10000000 >> select) : 8'd0;
11 endmodule
12
```



실습 8 / 1X8 DEMULTIPLEXER / 코드

src/combinational/demux1to8.v

L13-13

```
13 `default_nettype wire
```



실습 9 / 크기 비교기

조건	o[2:0]
A>B	100
A=B	010
A<B	001

- 두 4비트 unsigned 입력을 비교한다. 세 조건 중 하나만 참이다.
- 같음은 ==, 대소는 > 또는 <다. =는 비교가 아니라 대입이다.
- A=0/B=15, A=B, A=15/B=0의 경계를 먼저 확인한다.



실습 9 / 핀 참고

원래 예제의 포트 이름과 패키지 핀. 새 시뮬레이션 탑에는 적용하지 않는다.

포트	핀	포트	핀
a[3]	Y1	o[2]	L4
b[1]	V4	b[3]	W4
a[2]	W3	o[1]	M4
b[0]	U4	b[2]	W1
a[1]	U2	o[0]	M2
a[0]	T1		



실습 9 / 크기 비교기 / 시뮬레이션 확인

확인할 계층

tb_legacy.dut.u_comb

- a[3:0], b[3:0]에 (0,15), (7,7), (15,0)을 넣는다.
- 새 코드에서는 result[27]이 A>B, result[28]이 A=B다. 원표의 3비트 출력과 구분한다.
- 입력을 바꾸기 전에 예상 출력을 적고, VCD의 값과 비교한다.



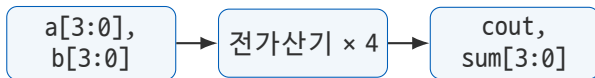
실습 9 / 크기 비교기 / 코드 연결

src/combinational/lab_combinational.v

전체 코드는 p. 43부터 수록했다.

이 실습의 입력·출력은 앞의 확인 항목과 함께 읽는다.

실습 13 / 4비트 덧셈 회로



- $15+15=30$ 이므로 결과는 carry를 포함해 5비트가 필요하다.
- 하위 비트의 carry가 다음 비트로 전달되는 리플 캐리 구조다.
- $0+0$, $15+1$, $15+15$ 를 확인한 다음 256가지 입력을 검사한다.

실습 13 / 핀 참고

원래 예제의 포트 이름과 패키지 핀. 새 시뮬레이션 탑에는 적용하지 않는다.

포트	핀	포트	핀
a[3]	Y1	s[1]	N7
cout	L4	b[3]	W4
a[2]	W3	s[0]	M7
s[3]	M4	b[2]	W1
a[1]	U2	b[1]	V4
s[2]	M2	b[0]	U4
a[0]	T1		



실습 13 / 4비트 덧셈 / 시뮬레이션 확인

확인할 계층

tb_legacy.dut.u_comb

- $a[3:0], b[3:0]$ 의 256가지 조합을 검사한다.
- $15+1$ 에서 $sum=10000$ 이다. $carry[0]$ 부터 $carry[3]$ 까지 연결을 추적한다.
- 입력을 바꾸기 전에 예상 출력을 적고, VCD의 값과 비교한다.



실습 13 / 4비트 덧셈 / 코드 연결

src/combinational/lab_combinational.v

전체 코드는 p. 43부터 수록했다.

이 실습의 입력·출력은 앞의 확인 항목과 함께 읽는다.

실습 14 / 4비트 뺄셈 회로

a	b	borrow	d[3:0]
0	0	0	0000
5	3	0	0010
0	1	1	1111
3	5	1	1110

- $a-b$ 가 음수가 되면 자리빌림이 발생한다. 하위 결과는 modulo 16이다.
- 새 코드의 difference는 0을 앞에 붙인 두 입력의 5비트 차다. 최상위 비트가 빌림을 나타낸다.
- borrow와 덧셈 carry의 의미는 다르다. 부호 있는 정수 해석과 구분한다.

실습 14 / 핀 참고

원래 예제의 포트 이름과 패키지 핀. 새 시뮬레이션 탑에는 적용하지 않는다.

포트	핀	포트	핀
a[3]	Y1	d[1]	N7
Bor	L4	b[3]	W4
a[2]	W3	d[0]	M7
d[3]	M4	b[2]	W1
a[1]	U2	b[1]	V4
d[2]	M2	b[0]	U4
a[0]	T1		



실습 14 / 4비트 뺄셈 / 시뮬레이션 확인

확인할 계층

tb_legacy.dut.u_comb

- $(a,b)=(5,3), (3,5), (0,1)$ 을 넣는다.
- difference는 5비트다. 3-5는 11110이며 하위 4비트와 빌림을 따로 해석한다.
- 입력을 바꾸기 전에 예상 출력을 적고, VCD의 값과 비교한다.



실습 14 / 4비트 뱀셈 / 코드 연결

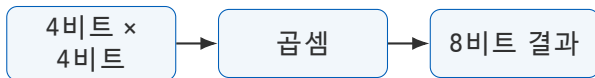
src/combinational/lab_combinational.v

전체 코드는 p. 43부터 수록했다.

이 실습의 입력·출력은 앞의 확인 항목과 함께 읽는다.



실습 15 / 4비트 곱셈 회로



- $15 \times 15 = 225$ 이므로 8비트 출력이 필요하다.
- 결과를 4비트로 선언하면 상위 비트가 사라진다.
- 0, 1을 곱한 경우와 15×15 를 포함하여 전수 검사한다.

실습 15 / 핀 참고

원래 예제의 포트 이름과 패키지 핀. 새 시뮬레이션 탑에는 적용하지 않는다.

포트	핀	포트	핀
a[3]	Y1	b[3]	W4
m[7]	L4	m[3]	M7
a[2]	W3	b[2]	W1
m[6]	M4	m[2]	M3
a[1]	U2	b[1]	V4
m[5]	M2	m[1]	M1
a[0]	T1	b[0]	U4
m[4]	N7	m[0]	N5



실습 15 / 4비트 곱셈 / 시뮬레이션 확인

확인할 계층

tb_legacy.dut.u_comb

- 0,1,15를 포함한 4비트 입력 두 개의 256가지 조합을 검사한다.
- 15 곱하기 15는 product=225(11100001)다. 8비트 출력 폭을 확인한다.
- 입력을 바꾸기 전에 예상 출력을 적고, VCD의 값과 비교한다.



실습 15 / 4비트 곱셈 / 코드 연결

src/combinational/lab_combinational.v

전체 코드는 p. 43부터 수록했다.

이 실습의 입력·출력은 앞의 확인 항목과 함께 읽는다.



계층적 가산기 검증

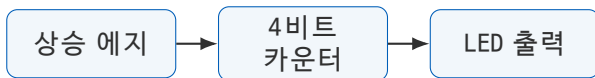
- 반가산기 2개로 전가산기를 만들고, 전가산기 4개로 4비트 덧셈기를 만든다.
- 4비트 입력 두 개의 256가지 조합을 전수 검사한다.
- 최상위 carry까지 포함한 5비트 결과를 비교한다.

순차회로 / 구성

2. 순차회로

첫 순차회로: 카운터	81
실습 16 / 카운터 회로 설계	86
실습 17 / 클럭 분주 회로 설계	90
실습 18 / 스텝 모터 제어 회로 설계	97
실습 26 / 데이터의 저장과 이동	102
실습 27 / 시프트 레지스터 구현	108
실습 28 / PISO 제어	113

순차회로 / 회로와 동작



- reset이 1이면 0으로 초기화하고 상승 에지마다 1씩 증가한다.
- 4비트이므로 15 다음은 0이다. 입력 변화와 상태 갱신 시점을 구분한다.
- 원자료는 버튼 클록을 사용한다. 새 코드는 기준 클록과 enable을 분리한다.

순차회로 / 핀 참고

원래 예제의 포트 이름과 패키지 핀. 새 시뮬레이션 탑에는 적용하지 않는다.

포트	핀	포트	핀
clk	K4	q[2]	M4
rst	N8	q[1]	M2
q[3]	L4	q[0]	N7

설명 p.4와 핀표 p.28의 clk/rst 스위치 지정이 서로 다르다. 아래는 핀표 기준이다.



첫 순차회로: 카운터 / 시뮬레이션 확인

확인할 계층

tb_legacy.dut.u_seq.u_counter

- rst=1인 상승 에지에서 count=0. rst=0, enable=1, down=0으로 증가시킨다.
- count가 15 다음에 0으로 돌아오는지, enable=0이면 유지되는지 확인한다.
- 입력을 바꾸기 전에 예상 출력을 적고, VCD의 값과 비교한다.



첫 순차회로: 카운터 / 코드

src/sequential/counter4.v

L1-12

```
1 `timescale 1ns/1ps
2 `default_nettype none
3
4 // 4비트 카운터: enable로 정지, down으로 증가/감소를 선택한다.
5 module counter4 (
6     input wire clk,
7     input wire rst,
8     input wire enable,
9     input wire down,
10    output reg [3:0] count
11 );
12    always @(posedge clk) begin
```



첫 순차회로: 카운터 / 코드

src/sequential/counter4.v

L13-21

```
13         if (rst) count <= 4'd0;
14         else if (enable) begin
15             if (down) count <= count - 1'b1;
16             else count <= count + 1'b1;
17         end
18     end
19 endmodule
20
21 `default_nettype wire
```



실습 16 / 카운터 회로 설계

조건	다음 count
reset=1	0
enable=0	현재 값
down=0	count+1
down=1	count-1

- 새 코드의 우선순위는 reset → enable → 방향이다.
- 15에서 증가하면 0, 0에서 감소하면 15로 순환한다.
- 저장된 값은 상승 에지에서 바뀐다. 조합회로와 비교한다.

실습 16 / 핀 참고

원래 예제의 포트 이름과 패키지 핀. 새 시뮬레이션 탑에는 적용하지 않는다.

포트	핀	포트	핀
rst	K4	dnup	Y1
q[3]	L4	q[1]	M2
clk	N8	q[0]	N7
q[2]	M4		

실습 16 / 카운터 / 시뮬레이션 확인

확인할 계층

tb_legacy.dut.u_seq.u_counter

- enable=1에서 down을 0과 1로 바꾼다.
- 증가 15→0, 감소 0→15와 enable=0에서의 유지를 확인한다.
- 입력을 바꾸기 전에 예상 출력을 적고, VCD의 값과 비교한다.



실습 16 / 카운터 / 코드 연결

src/sequential/counter4.v

전체 코드는 p. 84부터 수록했다.

이 실습의 입력·출력은 앞의 확인 항목과 함께 읽는다.



실습 17 / 클럭 분주 회로 설계

입력	출력	반전까지 클럭 수
100 Hz	100 Hz	직결
100 Hz	50 Hz	1
100 Hz	10 Hz	5
100 Hz	2 Hz	25

- N클럭마다 출력이 반전되면 $f_{\text{out}} = f_{\text{in}}/(2N)$ 이다.
- 증가 카운터의 $q[k]$ 는 입력 주파수를 $2^{(k+1)}$ 로 나눈 파형이다.
- 새 코드는 짧은 시뮬레이션 주기를 쓴다. 실제 주파수 설정과 구분한다.

실습 17 / 핀 참고

원래 예제의 포트 이름과 패키지 핀. 새 시뮬레이션 탭에는 적용하지 않는다.

포트	핀	포트	핀
rst	K4	o_50	M4
o_100	L4	o_10	M2
clk	B6	o_2	N7



실습 17 / 클럭 분주 / 시뮬레이션 확인

확인할 계층

tb_legacy.dut.u_seq.u_divider

- reset 후 enable=1로 두고 입력 clk와 분주 출력을 함께 본다.
- div2, div10, div50의 주기는 각각 2,10,50 입력 클록이다. direct_clock은 clk 직결이다.
- 입력을 바꾸기 전에 예상 출력을 적고, VCD의 값과 비교한다.



실습 17 / 클럭 분주 / 코드

src/sequential/clock_divider.v

L1-12

```
1  `timescale 1ns/1ps
2  `default_nettype none
3
4  // 실습 17: 100 Hz 입력이라면 100/50/10/2 Hz 파형.
5  // enable은 실행 여부일 뿐 새 클럭을 만들지 않는다.
6  module clock_divider (
7      input wire clk,
8      input wire rst,
9      input wire enable,
10     output wire direct_clock,
11     output reg div2,
12     output reg div10,
```



실습 17 / 클럭 분주 / 코드

src/sequential/clock_divider.v

L13-24

```
13     output reg div50
14 );
15     reg [2:0] count5;
16     reg [4:0] count25;
17     assign direct_clock = clk;
18     always @(posedge clk) begin
19         if (rst) begin
20             div2 <= 1'b0;
21             div10 <= 1'b0;
22             div50 <= 1'b0;
23             count5 <= 3'd0;
24             count25 <= 5'd0;
```



실습 17 / 클럭 분주 / 코드

src/sequential/clock_divider.v

L25-34

```
25         end else if (enable) begin
26             div2 <= ~div2;
27             if (count5 == 3'd4) begin
28                 count5 <= 3'd0;
29                 div10 <= ~div10;
30             end else count5 <= count5 + 1'b1;
31             if (count25 == 5'd24) begin
32                 count25 <= 5'd0;
33                 div50 <= ~div50;
34             end else count25 <= count25 + 1'b1;
```



실습 17 / 클럭 분주 / 코드

src/sequential/clock_divider.v

L35-39

```
35          end
36      end
37  endmodule
38
39  `default_nettype wire
```



실습 18 / 스텝 모터 제어 회로 설계



- 네 위상을 순환시키는 2상 여자 패턴이다. 두 출력이 동시에 1이다.
- 새 stepper_sequence는 enable마다 비트를 회전시킨다.
- 회전 방향은 위상 순서에 달려 있다. 실제 전류 구동과 기계 동작은 범위 밖이다.

실습 18 / 핀 참고

원래 예제의 포트 이름과 패키지 핀. 새 시뮬레이션 탑에는 적용하지 않는다.

포트	핀	포트	핀
rst	K4	stepmotor[2]	Y22
stepmotor[3]	Y20	stepmotor[1]	AA20
clk	B6	stepmotor[0]	AA21



실습 18 / 스텝 모터 제어 / 시뮬레이션 확인

확인할 계층

tb_legacy.dut.u_waveform.u_motor

- enable=1인 상승 에지마다 phase를 확인한다.
- 1001→0011→0110→1100→1001 순환이며 enable=0이면 유지된다.
- 입력을 바꾸기 전에 예상 출력을 적고, VCD의 값과 비교한다.



실습 18 / 스텝 모터 제어 / 코드

src/waveform/stepper_sequence.v

L1-12

```
1  `timescale 1ns/1ps
2  `default_nettype none
3
4  // 스텝 모터의 4개 여자 위상을 순환시키는 교육용 모델.
5  module stepper_sequence (
6      input wire clk,
7      input wire rst,
8      input wire enable,
9      output reg [3:0] phase
10 );
11     always @(posedge clk) begin
12         if (rst) phase <= 4'b1001;
```



실습 18 / 스텝 모터 제어 / 코드

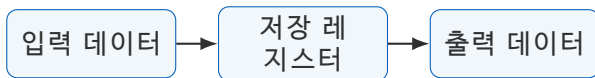
src/waveform/stepper_sequence.v

L13-17

```
13         else if (enable) phase <= {phase[2:0], phase[3]};
14     end
15 endmodule
16
17 `default_nettype wire
```



실습 26 / 데이터의 저장과 이동



- 원자료는 저장 버튼과 Load 버튼을 분리한 4비트 저장 예시다.
- 레지스터는 예지에서 입력을 기억하고 그 뒤 입력이 달라져도 유지한다.
- register4의 save는 저장, show는 출력 갱신이다. 동시에 1이면 이전 저장값이 출력된다.

실습 26 / 핀 참고

원래 예제의 포트 이름과 패키지 핀. 새 시뮬레이션 탑에는 적용하지 않는다.

포트	핀	포트	핀
sw_s	K4	o[2]	M4
sw_l	N8	i[1]	U2
i[3]	Y1	o[1]	M2
o[3]	L4	i[0]	T1
i[2]	W3	o[0]	N7



실습 26 / 데이터의 저장과 이동 / 시뮬레이션 확인

확인할 계층

tb_legacy.dut.u_seq.u_register4

- save로 data를 저장한 뒤 data를 바꾸고 show를 준다.
- 출력은 저장값이다. save와 show가 동시에 1이면 출력에는 이전 stored가 반영된다.
- 입력을 바꾸기 전에 예상 출력을 적고, VCD의 값과 비교한다.



실습 26 / 데이터의 저장과 이동 / 코드

src/sequential/register4.v

L1-12

```
1 `timescale 1ns/1ps
2 `default_nettype none
3
4 // 실습 26: 입력을 저장한 뒤, 별도 명령으로 저장값을 출력한다.
5 // save와 show가 동시에 1이면 출력에는 이전 저장값이 나온다.
6 module register4 (
7     input wire clk,
8     input wire rst,
9     input wire save,
10    input wire show,
11    input wire [3:0] data,
12    output reg [3:0] stored,
```



실습 26 / 데이터의 저장과 이동 / 코드

src/sequential/register4.v

L13-24

```
13     output reg [3:0] value
14 );
15     always @(posedge clk) begin
16         if (rst) begin
17             stored <= 4'd0;
18             value <= 4'd0;
19         end else begin
20             if (save) stored <= data;
21             if (show) value <= stored;
22         end
23     end
24 endmodule
```



실습 26 / 데이터의 저장과 이동 / 코드

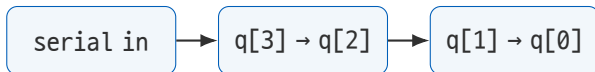
src/sequential/register4.v

L25-26

```
25  
26 `default_nettype wire
```



실습 27 / 시프트 레지스터 구현



- 원자료는 q[3]에서 q[0] 방향으로 이동한다.
- shift_register4는 {serial_in, value[3:1]}로 MSB에 새 비트를 넣는다.
- 0에서 입력 1, 0을 차례로 수락하면 1000, 0100이다. 8비트 보충 예제와 방향을 구분한다.

실습 27 / 핀 참고

원래 예제의 포트 이름과 패키지 핀. 새 시뮬레이션 탑에는 적용하지 않는다.

포트	핀	포트	핀
i	K4	o[2]	M4
o[3]	L4	o[1]	M2
clk	N8	o[0]	N7



실습 27 / 시프트 레지스터 / 시뮬레이션 확인

확인할 계층

tb_legacy.dut.u_seq.u_shift4

- 초기값 0000에서 serial_in=1,0을 차례로 수락시킨다.
- value는 1000,0100이다. enable=0일 때에는 이동하지 않는다.
- 입력을 바꾸기 전에 예상 출력을 적고, VCD의 값과 비교한다.



실습 27 / 시프트 레지스터 / 코드

src/sequential/shift_register4.v

L1-12

```
1  `timescale 1ns/1ps
2  `default_nettype none
3
4  // 실습 27: 새 비트는 MSB에 들어가고 q[3] -> q[0]으로 이동.
5  module shift_register4 (
6      input wire clk,
7      input wire rst,
8      input wire enable,
9      input wire serial_in,
10     output reg [3:0] value
11 );
12     always @(posedge clk) begin
```



실습 27 / 시프트 레지스터 / 코드

src/sequential/shift_register4.v

L13-18

```
13         if (rst) value <= 4'd0;
14         else if (enable) value <= {serial_in, value[3:1]};
15     end
16 endmodule
17
18 `default_nettype wire
```



실습 28 / PISO 제어

동작	레지스터	직렬 출력
병렬 적재	1010	1
1회 이동	0100	0
2회 이동	1000	1
3회 이동	0000	0

- PISO는 병렬 입력·직렬 출력이다.
표는 MSB-first 예시다.
- 출력 비트를 읽은 뒤 시프트해야 첫 비트를 놓치지 않는다.
- 원자료 shldn=1은 적재, 0은 출력이다.
새 코드는 load와 enable을 분리한다.

실습 28 / 핀 참고

원래 예제의 포트 이름과 패키지 핀. 새 시뮬레이션 탑에는 적용하지 않는다.

포트	핀	포트	핀
clk	K4	d[3]	Y1
d[1]	U2	d[2]	W3
shldn	N8	q	L4
d[0]	T1		

실습 28 / PISO 제어 / 시뮬레이션 확인

확인할 계층

tb_legacy.dut.u_seq.u_piso4

- load=1로 1010을 적재한 뒤 load=0, enable=1로 이동한다.
- serial_out을 이동 전에 읽으면 1,0,1,0이다. load가 enable보다 우선한다.
- 입력을 바꾸기 전에 예상 출력을 적고, VCD의 값과 비교한다.



실습 28 / PISO 제어 / 코드

src/sequential/piso4.v

L1-12

```
1  `timescale 1ns/1ps
2  `default_nettype none
3
4  // 실습 28: 병렬 적재 후 MSB-first 직렬 출력. load 우선.
5  module piso4 (
6      input wire clk,
7      input wire rst,
8      input wire load,
9      input wire enable,
10     input wire [3:0] data,
11     output reg [3:0] value,
12     output wire serial_out
```



실습 28 / PISO 제어 / 코드

src/sequential/piso4.v

L13-22

```
13 );
14     always @(posedge clk) begin
15         if (rst) value <= 4'd0;
16         else if (load) value <= data;
17         else if (enable) value <= {value[2:0], 1'b0};
18     end
19     assign serial_out = value[3];
20 endmodule
21
22 `default_nettype wire
```



표시 장치 / 구성

3. 표시 장치

실습 10 / BCD-2진 코드 변환기	119
실습 11 / 7-SEGMENT DECODER 설계	124
실습 12 / 7-SEGMENT ARRAY 제어	130
실습 32 / TEXT LCD 제어	135



실습 10 / BCD-2진 코드 변환기

십의 자리	일의 자리	2진수 결과
0	0	0000000
0	9	0001001
1	0	0001010
4	2	0101010
9	9	1100011

- 기존 제목의 ‘2비트’는 ‘2진 코드’로 바로잡는다. 결과는 7비트다.
- $\text{bin} = 10 \times \text{ten} + \text{one}$. 각 BCD 자리는 0 9만 유효하다.
- `bcd_to_binary`는 BCD를 이진수로 바꾼다. 자리값이 10 이상이면 `valid=0`, `binary=0`이다.

실습 10 / 핀 참고

원래 예제의 포트 이름과 패키지 핀. 새 시뮬레이션 탑에는 적용하지 않는다.

포트	핀	포트	핀
ten[3]	Y1	one[3]	W4
bin[6]	L4	bin[2]	M7
ten[2]	W3	one[2]	W1
bin[5]	M4	bin[1]	M3
ten[1]	U2	one[1]	V4
bin[4]	M2	bin[0]	M1
ten[0]	T1	one[0]	U4
bin[3]	N7		



실습 10 / BCD-2진 코드 변환기 / 시뮬레이션 확인

확인할 계층

tb_legacy.dut.u_display.u_bcd

- tens=4, ones=2이면 binary=42, valid=1이다.
- tens 또는 ones가 10 이상이면 valid=0, binary=0이다.
- 입력을 바꾸기 전에 예상 출력을 적고, VCD의 값과 비교한다.



실습 10 / BCD-2진 코드 변환기 / 코드

src/display/bcd_to_binary.v

L1-12

```
1  `timescale 1ns/1ps
2  `default_nettype none
3
4  // 실습 10: 십의 자리와 일의 자리 BCD를 0~99의 이진수로 변환.
5  module bcd_to_binary (
6      input wire [3:0] tens,
7      input wire [3:0] ones,
8      output wire [6:0] binary,
9      output wire valid
10 );
11     assign valid = (tens <= 4'd9) && (ones <= 4'd9);
12     assign binary = valid ? ({3'b000, tens} * 7'd10 + {3'b000, ones}) : 7'd0;
```



실습 10 / BCD-2진 코드 변환기 / 코드

src/display/bcd_to_binary.v

L13-15

```
13 endmodule
14
15 `default_nettype wire
```



실습 11 / 7-SEGMENT DECODER 설계

숫자	abcdefg
0	1111110
1	0110000
2	1101101
3	1111001
4	0110011
5	1011011
6	1011111
7	1110000
8	1111111
9	1111011

- 1은 점등이다. a g 순서와 소수점 dp를 먼저 정의한다.
- 원자료의 5·6·7 BCD 입력 오타를 0101·0110·0111로 바로잡았다.
- 새 hex7seg는 0 F를 표시하며 {a,b,c,d,e,f,g,dp} 순서다.

실습 11 / 핀 참고

원래 예제의 포트 이름과 패키지 핀. 새 시뮬레이션 탑에는 적용하지 않는다.

포트	핀	포트	핀
bcd[3]	Y1	bcd[0]	Y5
seg_data[5]	N4	seg_data[2]	M5
bcd[2]	W3	seg_data[7]	P1
seg_data[4]	N3	seg_data[2]	M4
bcd[1]	Y4	seg_data[6]	N5
seg_data[3]	N1	seg_data[0]	M3

원표에 중복 포트/핀 표기가 있다. 그대로 기록했으며 실제 배선값으로 확정하지 않는다.
seg_data[2]가 두 번 나타나는 원표의 인덱스 오류를 임의 수정하지 않았다.



실습 11 / 7-SEGMENT DECODER / 시뮬레이션 확인

확인할 계층

tb_legacy.dut.u_display.u_manual.u_decoder

- value=0,1,8,F를 차례로 넣는다.
- segments는 abcdefg,dp 순서다. 8의 출력은 11111110이며 dp는 0이다.
- 입력을 바꾸기 전에 예상 출력을 적고, VCD의 값과 비교한다.



실습 11 / 7-SEGMENT DECODER / 코드

src/display/hex7seg.v

L1-12

```
1  `timescale 1ns/1ps
2  `default_nettype none
3
4  // 7세그먼트 디코더. 1 = 점등, 비트 순서 = {a,b,c,d,e,f,g,dp}.
5  module hex7seg (
6      input wire [3:0] value,
7      output reg [7:0] segments
8  );
9      always @* begin
10         case (value)
11             4'h0: segments = 8'hfc;
12             4'h1: segments = 8'h60;
```



실습 11 / 7-SEGMENT DECODER / 코드

src/display/hex7seg.v

L13-24

```
13      4'h2: segments = 8'hda;
14      4'h3: segments = 8'hf2;
15      4'h4: segments = 8'h66;
16      4'h5: segments = 8'hb6;
17      4'h6: segments = 8'hbe;
18      4'h7: segments = 8'he0;
19      4'h8: segments = 8'hfe;
20      4'h9: segments = 8'hf6;
21      4'ha: segments = 8'hee;
22      4'hb: segments = 8'h3e;
23      4'hc: segments = 8'h9c;
24      4'hd: segments = 8'h7a;
```



실습 11 / 7-SEGMENT DECODER / 코드

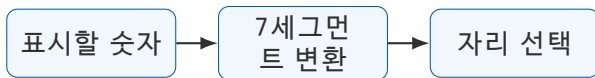
src/display/hex7seg.v

L25-32

```
25         4'he: segments = 8'h9e;
26         4'hf: segments = 8'h8e;
27         default: segments = 8'h00;
28     endcase
29 end
30 endmodule
31
32 `default_nettype wire
```



실습 12 / 7-SEGMENT ARRAY 제어



- segment 데이터와 자리 선택 신호는 다른 역할이다.
- segment_select8은 position=0 7로 한 자리를 선택한다. 버튼 입력 처리는 포함하지 않는다.
- 선택한 자리의 숫자와 segment 데이터가 맞는지 확인한다.

실습 12 / 핀 참고

원래 예제의 포트 이름과 패키지 핀. 새 시뮬레이션 탭에는 적용하지 않는다.

포트	핀	포트	핀
bcd[3]	Y1	sw[0]	N4
seg_com[3]	L6	seg_data[5]	E2
bcd[2]	W3	seg_data[4]	E4
seg_com[2]	K1	seg_com[7]	H4
bcd[1]	Y4	seg_data[3]	J1
seg_com[1]	K3	seg_com[6]	H6
bcd[0]	Y5	seg_data[2]	J3
seg_com[0]	K5	seg_com[5]	G1
sw[2]	K4	seg_data[1]	J7
seg_data[7]	F1	seg_com[4]	G3
sw[1]	N8	seg_data[0]	H2
seg_data[6]	F5		

실습 12 / 7-SEGMENT ARRAY 제어 / 시뮬레이션 확인

확인할 계층

tb_legacy.dut.u_display.u_manual

- digit를 고정하고 position을 0부터 7까지 바꾼다.
- digit_select만 이동하고 segments는 유지된다. 두 신호의 역할을 분리해서 본다.
- 입력을 바꾸기 전에 예상 출력을 적고, VCD의 값과 비교한다.



실습 12 / 7-SEGMENT ARRAY 제어 / 코드

src/display/segment_select8.v

L1-12

```
1  `timescale 1ns/1ps
2  `default_nettype none
3
4  // 실습 12: 선택한 자리만 켜다. 1=활성으로 정규화한 교육용 출력.
5  module segment_select8 (
6      input wire [2:0] position,
7      input wire [3:0] digit,
8      output wire [7:0] digit_select,
9      output wire [7:0] segments
10 );
11     assign digit_select = 8'b00000001 << position;
12     hex7seg u_decoder (
```



실습 12 / 7-SEGMENT ARRAY 제어 / 코드

src/display/segment_select8.v

L13-18

```
13         .value(digit),  
14         .segments(segments)  
15     );  
16 endmodule  
17  
18 `default_nettype wire
```



실습 32 / TEXT LCD 제어

단계	RS	data	의미
0	0	38h	8비트·2줄
1	0	0Ch	표시 켜기
2	0	06h	주소 증가
3	0	80h	첫 줄 시작
4	1	48h	문자 H

- RS=0은 명령, RS=1은 문자 데이터다.
E 펄스로 전달 시점을 알린다.
- 전송 순서는 데이터 설정 → E=1 → E=0
이다. 명령 4개 뒤 Hello World의 11
문자를 보낸다.
- 문자열 끝에서 첫 줄 주소 명령으로
돌아간다. 실제 LCD 초기화 대기
시간은 범위 밖이다.

실습 32 / 핀 참고

원래 예제의 포트 이름과 패키지 핀. 새 시뮬레이션 탑에는 적용하지 않는다.

포트	핀	포트	핀
rst	K4	lcd_data[6]	C1
lcd_data[3]	D4	lcd_e	A6
clk	B6	lcd_data[5]	C5
lcd_data[2]	C3	lcd_rs	G6
lcd_data[1]	B2	lcd_data[4]	A2
lcd_data[7]	D1	lcd_rw	D6
lcd_data[0]	A4		

실습 32 / TEXT LCD 제어 / 시뮬레이션 확인

확인할 계층

tb_legacy.dut.u_display.u_lcd

- data,rs,e를 함께 본다. 명령 38h,0Ch,06h,80h 뒤 문자열이 나온다.
- E의 하강 에지에서 Hello World 11문자를 확인한다. 데이터 설정과 전송 에지를 구분한다.
- 입력을 바꾸기 전에 예상 출력을 적고, VCD의 값과 비교한다.



실습 32 / TEXT LCD 제어 / 코드

src/display/lcd_sequence.v

L1-12

```
1  `timescale 1ns/1ps
2  `default_nettype none
3
4  // LCD 명령/문자 전송의 순서만 보여 주는 교육용 모델.
5  // 실제 LCD의 전원 투입 대기 시간이나 전기적 타이밍은 다루지 않는다.
6  module lcd_sequence (
7      input wire clk,
8      input wire rst,
9      input wire tick,
10     output reg [7:0] data,
11     output reg e,
12     output reg rs
```



실습 32 / TEXT LCD 제어 / 코드

src/display/lcd_sequence.v

L13-24

```
13 );  
14     reg [3:0] index;  
15     reg [1:0] phase;  
16     always @(posedge clk) begin  
17         if (rst) begin  
18             index <= 0;  
19             phase <= 0;  
20             e <= 0;  
21             rs <= 0;  
22             data <= 0;  
23         end else if (tick) begin  
24             case (phase)
```



실습 32 / TEXT LCD 제어 / 코드

src/display/lcd_sequence.v

L25-34

```
25         0: begin
26             e <= 0;
27             rs <= (index >= 4);
28             case (index)
29                 0: data <= 8'h38; // 8-bit, two lines
30                 1: data <= 8'h0c; // display on
31                 2: data <= 8'h06; // address increment
32                 3: data <= 8'h80; // first line
33                 4: data <= "H";
34                 5: data <= "e";
```



실습 32 / TEXT LCD 제어 / 코드

src/display/lcd_sequence.v

L35-46

```
35         6: data <= "l";
36         7: data <= "l";
37         8: data <= "o";
38         9: data <= " ";
39        10: data <= "W";
40        11: data <= "o";
41        12: data <= "r";
42        13: data <= "l";
43        14: data <= "d";
44        default: data <= " ";
45    endcase
46    phase <= 1;
```



실습 32 / TEXT LCD 제어 / 코드

src/display/lcd_sequence.v

L47-58

```
47         end
48         1: begin
49             e <= 1;
50             phase <= 2;
51         end
52         default: begin
53             e <= 0;
54             phase <= 0;
55             if (index == 14) index <= 3;
56             else index <= index + 1'b1;
57         end
58     endcase
```



실습 32 / TEXT LCD 제어 / 코드

src/display/lcd_sequence.v

L59-63

```
59         end
60     end
61 endmodule
62
63 `default_nettype wire
```

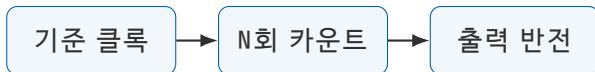


파형 생성과 구동 / 구성

4. 파형 생성과 구동

실습 19 / PIEZO 제어	145
실습 20 / PIEZO로 피아노 만들기	151
실습 21 / 시계 만들기	159
실습 22 / PWM을 이용한 LED 밝기 제어	167
실습 23 / FULL COLOR LED 순차 제어	172
실습 24 / FULL COLOR LED PWM 제어	177

실습 19 / PIEZO 제어



- 주기적으로 반전시키면 사각파가 된다. $f = f_{\text{clk}}/(2N)$.
- 원자료는 1 MHz 입력을 분주한다.
- 새 tone_generator의 반주기는 $16+2 \times \text{note}$ 클럭이다. 실제 음계가 아닌 원리 관찰용이다.

실습 19 / 핀 참고

원래 예제의 포트 이름과 패키지 핀. 새 시뮬레이션 탭에는 적용하지 않는다.

포트	핀	포트	핀
rst	K4	clk	B6
piezo	Y21		



실습 19 / PIEZO 제어 / 시뮬레이션 확인

확인할 계층

tb_legacy.dut.u_waveform.u_tone

- note를 고정하고 tone의 두 전환 사이 클록 수를 센다.
- 반주기는 16+2 곱하기 note 클록이다. 이것을 두 배 하면 전체 주기다.
- 입력을 바꾸기 전에 예상 출력을 적고, VCD의 값과 비교한다.



실습 19 / PIEZO 제어 / 코드

src/waveform/tone_generator.v

L1-12

```
1  `timescale 1ns/1ps
2  `default_nettype none
3
4  // 음원 분주 예시: 시뮬레이션용 짧은 주기이며 실제 음계가 아니다.
5  module tone_generator (
6      input wire clk,
7      input wire rst,
8      input wire enable,
9      input wire [2:0] note,
10     output reg tone
11 );
12     reg [7:0] count;
```



실습 19 / PIEZO 제어 / 코드

src/waveform/tone_generator.v

L13-24

```
13    wire [7:0] half_period = 8'd16 + {4'b0000, note, 1'b0};
14    always @(posedge clk) begin
15        if (rst) begin
16            count <= 0;
17            tone <= 0;
18        end
19        else if (enable) begin
20            if (count == half_period - 1'b1) begin
21                count <= 0;
22                tone <= ~tone;
23            end else count <= count + 1'b1;
24        end
```



실습 19 / PIEZO 제어 / 코드

src/waveform/tone_generator.v

L25-28

```
25     end
26 endmodule
27
28 `default_nettype wire
```



실습 20 / PIEZO로 피아노 만들기



- 키마다 다른 분주값을 선택하면 다른 높이의 음을 만든다.
- 여러 키의 우선순위와 아무 키도 없을 때의 무음을 정의한다.
- piano는 8키 중 MSB 우선으로 선택한다. 무입력은 무음이며 반주기 4+note는 관찰용 값이다.

실습 20 / 핀 참고

원래 예제의 포트 이름과 패키지 핀. 새 시뮬레이션 탑에는 적용하지 않는다.

포트	핀	포트	핀
rst	Y1	key[6]	N8
key[3]	P6	key[0]	J2
clk	B6	key[5]	N4
key[2]	N6	key[4]	N1
key[7]	K4	piezo	Y21
key[1]	L5		

실습 20 / PIEZO로 피아노 만들기 / 시뮬레이션 확인

확인할 계층

tb_legacy.dut.u_waveform.u_piano

- 한 키씩 누른 뒤 무입력과 동시 입력을 검사한다.
- MSB 키가 우선한다. 선택 음이 바뀌거나 무음에서 시작하면 위상을 초기화한다.
- 입력을 바꾸기 전에 예상 출력을 적고, VCD의 값과 비교한다.



실습 20 / PIEZO로 피아노 만들기 / 코드

src/waveform/piano.v

L1-12

```
1 `timescale 1ns/1ps
2 `default_nettype none
3
4 // 실습 20: MSB 키 우선. 무입력은 무음, 키 변경은 위상을 초기화.
5 // 반주기는 시뮬레이션용 값이며 실제 음계 주파수가 아니다.
6 module piano (
7     input wire clk,
8     input wire rst,
9     input wire enable,
10    input wire [7:0] keys,
11    output reg tone,
12    output reg active,
```



실습 20 / PIEZO로 피아노 만들기 / 코드

src/waveform/piano.v

L13-24

```
13     output reg [2:0] note
14 );
15     reg [4:0] count;
16     reg [2:0] previous_note;
17     reg was_active;
18     wire [4:0] half_period = 5'd4 + {2'b00, note};
19     always @* begin
20         active = !keys;
21         if      (keys[7]) note = 3'd0;
22         else if (keys[6]) note = 3'd1;
23         else if (keys[5]) note = 3'd2;
24         else if (keys[4]) note = 3'd3;
```



실습 20 / PIEZO로 피아노 만들기 / 코드

src/waveform/piano.v

L25-34

```
25         else if (keys[3]) note = 3'd4;
26         else if (keys[2]) note = 3'd5;
27         else if (keys[1]) note = 3'd6;
28         else           note = 3'd7;
29     end
30     always @(posedge clk) begin
31         if (rst) begin
32             count <= 5'd0;
33             tone <= 1'b0;
34             previous_note <= 3'd0;
```



실습 20 / PIEZO로 피아노 만들기 / 코드

src/waveform/piano.v

L35-46

```
35         was_active <= 1'b0;
36     end else if (enable) begin
37         previous_note <= note;
38         was_active <= active;
39         if (!active || !was_active || note != previous_note) begin
40             count <= 5'd0;
41             tone <= 1'b0;
42         end else if (count == half_period - 1'b1) begin
43             count <= 5'd0;
44             tone <= ~tone;
45         end else count <= count + 1'b1;
46     end
```



실습 20 / PIEZO로 피아노 만들기 / 코드

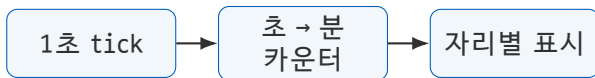
src/waveform/piano.v

L47-50

```
47     end
48 endmodule
49
50 `default_nettype wire
```



실습 21 / 시계 만들기



- 초가 59에서 넘어갈 때 분을 증가시킨다.
- 시간 계수와 표시 스캔은 다른 주기다. 같은 기준 클록에서 enable을 나눈다.
- watch_mmss에서 00:59→01:00, 59:59→00:00을 검사한다. tick 한 번을 1초로 간주한다.

실습 21 / 핀 참고

원래 예제의 포트 이름과 패키지 핀. 새 시뮬레이션 탑에는 적용하지 않는다.

포트	핀	포트	핀
rst	K4	seg_data[4]	E4
seg_com[1]	K3	seg_com[5]	G1
clk	B6	seg_data[3]	J1
seg_com[0]	K5	seg_com[4]	G3
seg_data[7]	F1	seg_data[2]	J3
seg_data[6]	F5	seg_com[3]	L6
seg_com[7]	H4	seg_data[2]	J7
seg_data[5]	E2	seg_com[2]	K1
seg_com[6]	H6	seg_data[0]	H2

원표에 중복 포트/핀 표기가 있다. 그대로 기록했으며 실제 배선값으로 확정하지 않는다.
seg_data[2]가 두 번 나타나는 원표의 인덱스 오류를 임의 수정하지 않았다.

실습 21 / 시계 만들기 / 시뮬레이션 확인

확인할 계층

tb_legacy.dut.u_waveform.u_watch

- tick=1인 상승 에지를 1초로 간주하여 초와 분을 증가시킨다.
- 00:59→01:00, 59:59→00:00을 확인한다. 자리 스캔은 tick과 별도로 계속 진행한다.
- 입력을 바꾸기 전에 예상 출력을 적고, VCD의 값과 비교한다.



실습 21 / 시계 만들기 / 코드

src/waveform/watch_mmss.v

L1-12

```
1  `timescale 1ns/1ps
2  `default_nettype none
3
4  // 실습 21: tick 하나를 1초로 간주한다. 59:59 다음은 00:00.
5  // 빠른 시뮬레이션을 위해 실제 1초 분주는 외부에 둔다.
6  module watch_mmss (
7      input wire clk,
8      input wire rst,
9      input wire tick,
10     output reg [5:0] seconds,
11     output reg [5:0] minutes,
12     output wire [7:0] segments,
```



실습 21 / 시계 만들기 / 코드

src/waveform/watch_mmss.v

L13-24

```
13     output wire [3:0] digit_select
14 );
15     reg [1:0] position;
16     reg [5:0] decimal_digit;
17     always @(posedge clk) begin
18         if (rst) begin
19             seconds <= 6'd0;
20             minutes <= 6'd0;
21             position <= 2'd0;
22         end else begin
23             position <= position + 1'b1;
24             if (tick) begin
```



실습 21 / 시계 만들기 / 코드

src/waveform/watch_mmss.v

L25-34

```
25             if (seconds == 6'd59) begin
26                 seconds <= 6'd0;
27                 minutes <= (minutes == 6'd59) ? 6'd0 : minutes + 1'b1;
28             end else seconds <= seconds + 1'b1;
29         end
30     end
31 end
32 always @* begin
33     case (position)
34         2'd0: decimal_digit = seconds % 6'd10;
```



실습 21 / 시계 만들기 / 코드

src/waveform/watch_mmss.v

L35-46

```
35         2'd1: decimal_digit = seconds / 6'd10;
36         2'd2: decimal_digit = minutes % 6'd10;
37         default: decimal_digit = minutes / 6'd10;
38     endcase
39 end
40 assign digit_select = 4'b0001 << position;
41 hex7seg u_decoder (
42     .value(decimal_digit[3:0]),
43     .segments(segments)
44 );
45 endmodule
46
```



실습 21 / 시계 만들기 / 코드

src/waveform/watch_mmss.v

L47-47

```
47 `default_nettype wire
```



실습 22 / PWM을 이용한 LED 밝기 제어



- PWM은 주기를 유지하고 1인 시간의 비율을 바꾼다.
- 새 pwm4는 phase=0 15와 duty를 비교한다. duty=4는 25%다.
- duty=0은 항상 0, duty=15는 15/16이다. 이 입력 폭에는 100% 설정이 없다.

실습 22 / 핀 참고

원래 예제의 포트 이름과 패키지 핀. 새 시뮬레이션 탑에는 적용하지 않는다.

포트	핀	포트	핀
rst	Y1	led[2]	M3
led[5]	M2	led[7]	L4
clk	B6	led[1]	M1
led[4]	N7	led[6]	M4
sw	K4	led[0]	N5
led[3]	M7		



실습 22 / PWM을 이용한 LED 밝기 제어 / 시뮬레이션 확인

확인할 계층

tb_legacy.dut.u_waveform.u_red

- duty를 0,4,8,15로 바꾸고 phase의 16클럭 구간을 센다.
- HIGH 클럭 수는 각각 0,4,8,15다. 15는 100%가 아니다.
- 입력을 바꾸기 전에 예상 출력을 적고, VCD의 값과 비교한다.



실습 22 / PWM을 이용한 LED 밝기 제어 / 코드

src/waveform/pwm4.v

L1-12

```
1 `timescale 1ns/1ps
2 `default_nettype none
3
4 // 16클록 주기 PWM: duty/16 비율로 출력이 1이 된다.
5 module pwm4 (
6     input wire clk,
7     input wire rst,
8     input wire enable,
9     input wire [3:0] duty,
10    output wire pulse
11 );
12    reg [3:0] phase;
```



실습 22 / PWM을 이용한 LED 밝기 제어 / 코드

src/waveform/pwm4.v

L13-20

```
13     always @(posedge clk) begin
14         if (rst) phase <= 0;
15         else if (enable) phase <= phase + 1'b1;
16     end
17     assign pulse = (phase < duty);
18 endmodule
19
20 `default_nettype wire
```



실습 23 / FULL COLOR LED 순차 제어

R	G	B	색
1	0	0	빨강
0	1	0	초록
0	0	1	파랑
1	1	0	노랑
1	1	1	흰색

- RGB 세 출력을 조합한다. 모두 0이면 꺼짐이다.
- 원자료는 버튼 입력마다 색 상태를 순환한다. 입력과 상태를 구분한다.
- 단순 ON/OFF 색 선택과 PWM 밝기 조절은 다르다.

실습 23 / 핀 참고

원래 예제의 포트 이름과 패키지 핀. 새 시뮬레이션 탑에는 적용하지 않는다.

포트	핀	포트	핀
sw	K4	cled_r[1]	P2
cled_g[2]	V1	cled_b[2]	W2
cled_g[1]	R7	cled_r[0]	R3
cled_r[3]	T2	cled_b[1]	R5
cled_g[0]	T6	cled_g[3]	U5
cled_r[2]	U1	cled_b[0]	T3
cled_b[3]	U3		



실습 23 / FULL COLOR LED 순차 제어 / 시뮬레이션 확인

확인할 계층

tb_legacy.dut.u_waveform.u_rgb

- reset 후 rgb=100이고 next_color마다 다음 색으로 이동한다.
- 100→010→001→100이다. 원표의 혼합색 조합은 보충 설명이며 이 코드는 세 색만 순환한다.
- 입력을 바꾸기 전에 예상 출력을 적고, VCD의 값과 비교한다.



실습 23 / FULL COLOR LED 순차 제어 / 코드

src/waveform/rgb_sequence.v

L1-12

```
1  `timescale 1ns/1ps
2  `default_nettype none
3
4  // 실습 23: next_color 한 펄스마다 빨강 -> 초록 -> 파랑 순환.
5  module rgb_sequence (
6      input wire clk,
7      input wire rst,
8      input wire next_color,
9      output reg [2:0] rgb
10 );
11     always @(posedge clk) begin
12         if (rst) rgb <= 3'b100;
```



실습 23 / FULL COLOR LED 순차 제어 / 코드

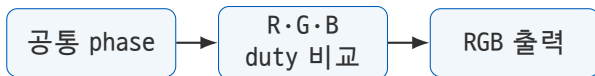
src/waveform/rgb_sequence.v

L13-17

```
13         else if (next_color) rgb <= {rgb[0], rgb[2:1]};  
14     end  
15 endmodule  
16  
17 `default_nettype wire
```



실습 24 / FULL COLOR LED PWM 제어



- 세 채널은 같은 주기, 서로 다른 duty를 갖는다.
- 새 코드는 빨강 가변, 초록 4/16, 파랑 12/16로 구성했다.
- 여기서는 디지털 비율을 검사한다. 사람이 느끼는 밝기가 duty와 완전히 비례하는 것은 아니다.

실습 24 / 핀 참고

원래 예제의 포트 이름과 패키지 핀. 새 시뮬레이션 탑에는 적용하지 않는다.

포트	핀	포트	핀
rst	Y1	cled_g[0]	T6
cled_r[0]	R3	cled_b[3]	U3
clk	B6	cled_r[3]	T2
cled_g[3]	U5	cled_b[2]	W2
sw_r	K4	cled_r[2]	U1
cled_g[2]	V1	cled_b[1]	R5
sw_g	N8	cled_r[1]	P2
cled_g[1]	R7	cled_b[0]	T3
sw_b	N4		

실습 24 / FULL COLOR LED PWM 제어 / 시뮬레이션 확인

확인할 계층

tb_legacy.dut.u_waveform

- 세 채널의 phase가 일치하는 상태에서 duty를 바꾼다.
- 빨강은 가변, 초록은 4/16, 파랑은 12/16의 HIGH 비율이다.
- 입력을 바꾸기 전에 예상 출력을 적고, VCD의 값과 비교한다.



실습 24 / FULL COLOR LED PWM 제어 / 코드

src/waveform/lab_waveform.v

L1-12

```
1 `timescale 1ns/1ps
2 `default_nettype none
3
4 // 파형 생성 묶음: RGB PWM, 음원, 모터 위상을 함께 관찰한다.
5 module lab_waveform (
6     input wire clk,
7     input wire rst,
8     input wire enable,
9     input wire [3:0] duty,
10    input wire [7:0] keys,
11    input wire [2:0] note,
12    output wire [7:0] result
```



실습 24 / FULL COLOR LED PWM 제어 / 코드

src/waveform/lab_waveform.v

L13-24

```
13 );  
14     wire red, green, blue, tone;  
15     wire [3:0] motor;  
16     wire piano_tone, piano_active;  
17     wire [2:0] piano_note, rgb_state;  
18     wire [5:0] seconds, minutes;  
19     wire [7:0] watch_segments;  
20     wire [3:0] watch_digits;  
21  
22     pwm4 u_red (  
23         .clk(clk),  
24         .rst(rst),
```



실습 24 / FULL COLOR LED PWM 제어 / 코드

src/waveform/lab_waveform.v

L25-29

```
25         .enable(enable),  
26         .duty(duty),  
27         .pulse(red)  
28     );  
29
```



실습 24 / FULL COLOR LED PWM 제어 / 코드

src/waveform/lab_waveform.v

L30-41

```
30     pwm4 u_green (  
31         .clk(clk),  
32         .rst(rst),  
33         .enable(enable),  
34         .duty(4'd4),  
35         .pulse(green)  
36     );  
37  
38     pwm4 u_blue (  
39         .clk(clk),  
40         .rst(rst),  
41         .enable(enable),
```



실습 24 / FULL COLOR LED PWM 제어 / 코드

src/waveform/lab_waveform.v

L42-53

```
42         .duty(4'd12),  
43         .pulse(blue)  
44     );  
45  
46     tone_generator u_tone (  
47         .clk(clk),  
48         .rst(rst),  
49         .enable(enable),  
50         .note(note),  
51         .tone(tone)  
52     );  
53
```



실습 24 / FULL COLOR LED PWM 제어 / 코드

src/waveform/lab_waveform.v

L54-62

```
54     stepper_sequence u_motor (  
55         .clk(clk),  
56         .rst(rst),  
57         .enable(enable),  
58         .phase(motor)  
59     );  
60  
61     assign result = {motor, tone, blue, green, red};  
62
```



실습 24 / FULL COLOR LED PWM 제어 / 코드

src/waveform/lab_waveform.v

L63-74

```
63     piano u_piano (  
64         .clk(clk),  
65         .rst(rst),  
66         .enable(enable),  
67         .keys(keys),  
68         .tone(piano_tone),  
69         .active(piano_active),  
70         .note(piano_note)  
71     );  
72  
73     watch_mmss u_watch (  
74         .clk(clk),
```



실습 24 / FULL COLOR LED PWM 제어 / 코드

src/waveform/lab_waveform.v

L75-86

```
75         .rst(rst),
76         .tick(enable),
77         .seconds(seconds),
78         .minutes(minutes),
79         .segments(watch_segments),
80         .digit_select(watch_digits)
81     );
82
83     rgb_sequence u_rgb (
84         .clk(clk),
85         .rst(rst),
86         .next_color(enable),
```



실습 24 / FULL COLOR LED PWM 제어 / 코드

src/waveform/lab_waveform.v

L87-92

```
87         .rgb(rgb_state)
88     );
89
90 endmodule
91
92 `default_nettype wire
```

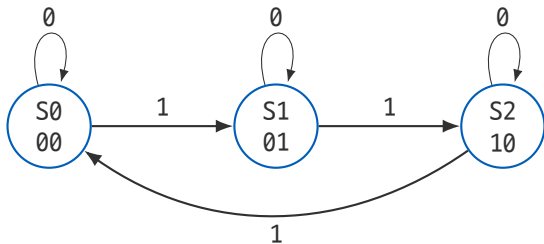


상태 머신 / 구성

5. 상태 머신

실습 29 / 무어 스테이트 머신 설계	190
실습 30 / 밀리 스테이트 머신 구현	195
응용 / 상태 머신 통합 제어	200

실습 29 / 무어 스테이트 머신 설계



- Moore 출력은 현재 상태로 결정된다. 원자료는 00/01/10 출력의 3상태 순환이다.
- 입력 0은 유지, 1은 다음 상태로 진행한다.
- moore_cycle0이 위 3상태를 구현한다. 보충 moore101은 별도의 패턴 검출기다.

실습 29 / 핀 참고

원래 예제의 포트 이름과 패키지 핀. 새 시뮬레이션 탭에는 적용하지 않는다.

포트	핀	포트	핀
rst	K4	x	N8
y[1]	L4	y[0]	M4



실습 29 / 무어 스테이트 머신 / 시뮬레이션 확인

확인할 계층

tb_legacy.dut.u_fsm.u_cycle

- enable=1에서 advance를 0과 1로 바꾼다.
- 0이면 상태 유지, 1이면 00→01→10→00이다. value는 현재 state와 같다.
- 입력을 바꾸기 전에 예상 출력을 적고, VCD의 값과 비교한다.



실습 29 / 무어 스테이트 머신 / 코드

src/fsm/moore_cycle.v

L1-12

```
1  `timescale 1ns/1ps
2  `default_nettype none
3
4  // 실습 29 원교안: 입력 0은 유지, 입력 1은 세 상태 순환.
5  module moore_cycle (
6      input wire clk,
7      input wire rst,
8      input wire enable,
9      input wire advance,
10     output reg [1:0] state,
11     output wire [1:0] value
12 );
```



실습 29 / 무어 스테이트 머신 / 코드

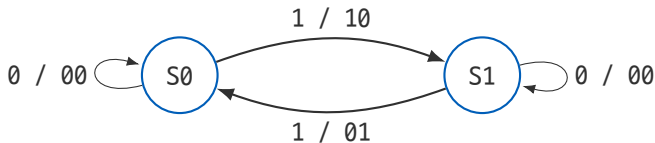
src/fsm/moore_cycle.v

L13-21

```
13     always @(posedge clk) begin
14         if (rst) state <= 2'd0;
15         else if (enable && advance)
16             state <= (state == 2'd2) ? 2'd0 : state + 1'b1;
17     end
18     assign value = state;
19 endmodule
20
21 `default_nettype wire
```



실습 30 / 밀리 스테이트 머신 구현



- 화살표는 입력 / 출력이다. Mealy 출력은 상태와 현재 입력에 의해 결정된다.
- 원자료는 입력 1마다 S0와 S1을 번갈아 이동하며 다른 값을 출력한다.
- mealy_toggle은 상태와 bit_in으로 즉시 출력을 정한다. enable은 상태 갱신만 허용한다.

실습 30 / 핀 참고

원래 예제의 포트 이름과 패키지 핀. 새 시뮬레이션 탑에는 적용하지 않는다.

포트	핀	포트	핀
rst	K4	x	N8
y[1]	L4	y[0]	M4

실습 30 / 밀리 스테이트 머신 / 시뮬레이션 확인

확인할 계층

tb_legacy.dut.u_fsm.u_toggle

- bit_in=0이면 출력 00, 1이면 상태에 따라 10 또는 01이다.
- enable=0으로 상태가 멈춰도 bit_in이 바뀌면 출력이 달라질 수 있다.
- 입력을 바꾸기 전에 예상 출력을 적고, VCD의 값과 비교한다.



실습 30 / 밀리 스테이트 머신 / 코드

src/fsm/mealy_toggle.v

L1-12

```
1  `timescale 1ns/1ps
2  `default_nettype none
3
4  // 실습 30 원교안: 출력은 상태와 입력에 의존한다.
5  module mealy_toggle (
6      input wire clk,
7      input wire rst,
8      input wire enable,
9      input wire bit_in,
10     output reg state,
11     output wire [1:0] value
12 );
```



실습 30 / 밀리 스테이트 머신 / 코드

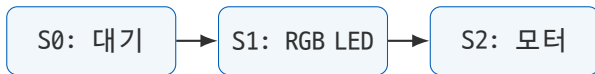
src/fsm/mealy_toggle.v

L13-20

```
13     always @(posedge clk) begin
14         if (rst) state <= 1'b0;
15         else if (enable && bit_in) state <= ~state;
16     end
17     assign value = !bit_in ? 2'b00 : (state ? 2'b01 : 2'b10);
18 endmodule
19
20 `default_nettype wire
```



응용 / 상태 머신 통합 제어



- 원자료 9.1절 p.27의 응용 과제다. 모드를 바꾸면 세 상태가 순환한다.
- 현재 상태에 따라 RGB 또는 모터 동작을 허용하는 구조로 나눈다.
- mode_controller는 next_mode로 모드를 바꾸고, 비활성 장치의 출력은 0으로 둔다.

응용 / 상태 머신 통합 제어 / 핀 참고 (1)

원래 예제의 포트 이름과 패키지 핀. 새 시뮬레이션 탑에는 적용하지 않는다.

포트	핀	포트	핀
clk	B6	seg_com[4]	G3
step_motor[3]	Y20	fled_r[2]	U1
rst	K4	seg_com[3]	L6
step_motor[2]	Y22	fled_r[1]	P2
mode	N8	seg_com[2]	K1
step_motor[1]	AA20	fled_r[0]	K4
step_motor[0]	AA21	seg_com[1]	K3
motor_sense	Y19	fled_g[3]	U5
seg_com[7]	H4	seg_com[0]	K5
seg_com[6]	H6	fled_g[2]	V1
seg_com[5]	G1	seg_data[7]	F1
fled_r[3]	T2	fled_g[1]	R7

원표에 중복 포트/핀 표기가 있다. 그대로 기록했으며 실제 배선값으로 확정하지 않는다.
fled_r[0]=K4는 rst와 충돌한다. 다른 RGB 예제의 R3와도 다르므로 원본 XDC 대조가 필요하다.
seg_data[2]가 두 번 나타나는 원표의 인덱스 오류를 임의 수정하지 않았다.



응용 / 상태 머신 통합 제어 / 핀 참고 (2)

원래 예제의 포트 이름과 패키지 핀. 새 시뮬레이션 탑에는 적용하지 않는다.

포트	핀	포트	핀
seg_data[6]	F5	seg_data[3]	J1
fled_g[0]	T6	fled_b[1]	R5
seg_data[5]	E2	seg_data[2]	J3
fled_b[3]	U3	fled_b[0]	T3
seg_data[4]	E4	seg_data[2]	J7
fled_b[2]	W2	seg_data[0]	H2

원표에 중복 포트/핀 표기가 있다. 그대로 기록했으며 실제 배선값으로 확정하지 않는다.
fled_r[0]=K4는 rst와 충돌한다. 다른 RGB 예제의 R3와도 다르므로 원본 XDC 대조가 필요하다.
seg_data[2]가 두 번 나타나는 원표의 인덱스 오류를 임의 수정하지 않았다.

응용 / 상태 머신 통합 제어 / 시뮬레이션 확인

확인할 계층

tb_legacy.dut.u_fsm.u_mode

- next_mode로 대기→RGB→모터→대기를 순환한다.
- RGB 모드에서는 motor=0, 모터 모드에서는 rgb=0이다. 장치 진행은 enable로 제어한다.
- 입력을 바꾸기 전에 예상 출력을 적고, VCD의 값과 비교한다.



응용 / 상태 머신 통합 제어 / 코드

src/fsm/mode_controller.v

L1-12

```
1  `timescale 1ns/1ps
2  `default_nettype none
3
4  // 응용: 대기 -> RGB -> 모터. 비활성 장치 출력은 0으로 둔다.
5  module mode_controller (
6      input wire clk,
7      input wire rst,
8      input wire enable,
9      input wire next_mode,
10     output reg [1:0] mode,
11     output wire [2:0] rgb,
12     output wire [3:0] motor
```



응용 / 상태 머신 통합 제어 / 코드

src/fsm/mode_controller.v

L13-24

```
13 );
14     wire [2:0] rgb_pattern;
15     wire [3:0] motor_pattern;
16     always @(posedge clk) begin
17         if (rst) mode <= 2'd0;
18         else if (next_mode) mode <= (mode == 2'd2) ? 2'd0 : mode + 1'b1;
19     end
20     rgb_sequence u_rgb (
21         .clk(clk),
22         .rst(rst),
23         .next_color(enable && mode == 2'd1),
24         .rgb(rgb_pattern)
```



응용 / 상태 머신 통합 제어 / 코드

src/fsm/mode_controller.v

L25-34

```
25     );
26     stepper_sequence u_motor (
27         .clk(clk),
28         .rst(rst),
29         .enable(enable && mode == 2'd2),
30         .phase(motor_pattern)
31     );
32     assign rgb = (mode == 2'd1) ? rgb_pattern : 3'b000;
33     assign motor = (mode == 2'd2) ? motor_pattern : 4'b0000;
34 endmodule
```



응용 / 상태 머신 통합 제어 / 코드

src/fsm/mode_controller.v

L35-36

```
35
```

```
36 `default_nettype wire
```



인터페이스와 통합 검증 / 구성

6. 인터페이스와 통합 검증

UART 8N1	209
통합 시뮬레이션	214
실습별 VCD 시간 구간	216
통합 계층과 보충 모듈 코드	219
단일 테스트벤치 전체 코드	270



UART 8N1

- 유희 상태는 1, 시작 비트는 0이다.
- 8비트를 최하위 비트부터 보내고 정지 비트 1로 프레임을 마친다.
- 송신 출력과 수신 입력을 연결한 루프백으로 256가지 바이트를 검사한다.
- 외부 보드 입력에는 동기화가 필요하다. 현재 예제는 동일 클록 도메인의 내부 루프백이다.

UART 8N1 / 시뮬레이션 확인

확인할 계층

tb_legacy.dut.u_interface

- start 펄스와 data를 넣고 tx,busy,received,valid를 관찰한다.
- 8N1, LSB-first다. valid가 1인 수신 완료 시점에서 received와 송신값을 비교한다.
- 입력을 바꾸기 전에 예상 출력을 적고, VCD의 값과 비교한다.

UART 8N1 / 코드

src/interface/lab_interface.v

L1-12

```
1  `timescale 1ns/1ps
2  `default_nettype none
3
4  // UART 내부 루프백: 외부 장비 없이 송신과 수신을 연결한다.
5  module lab_interface (
6      input wire clk,
7      input wire rst,
8      input wire start,
9      input wire [7:0] data,
10     output wire tx,
11     output wire [7:0] received,
12     output wire valid,
```



UART 8N1 / 코드

src/interface/lab_interface.v

L13-24

```
13     output wire busy
14 );
15
16     uart_transmitter u_tx (
17         .clk(clk),
18         .rst(rst),
19         .start(start),
20         .data(data),
21         .tx(tx),
22         .busy(busy)
23     );
24
```



UART 8N1 / 코드

src/interface/lab_interface.v

L25-34

```
25     uart_receiver u_rx (  
26         .clk(clk),  
27         .rst(rst),  
28         .rx(tx),  
29         .data(received),  
30         .valid(valid)  
31     );  
32 endmodule  
33  
34 `default_nettype wire
```



전체 시뮬레이션 실행

- `legacy_all_hdl.code-workspace`를 VS Code에서 연다.
- 빌드 태스크 Legacy: simulate all을 실행한다.
- `xvlog`, `xelab`, `xsim`을 순서대로 실행한다.
- 37개 설계 모듈, 테스트벤치 하나, 6개 계층. XSim 자동검사 20,727개 통과.
- `build/legacy.vcd`를 VaporView에서 열고 계층별 신호를 선택한다.



보드 검증과 시뮬레이션의 구분

- 새 코드의 reset은 동기식 active-high다. 원자료의 reset 방식과 구분한다.
- PWM·음원·LCD는 빠른 시뮬레이션을 위한 기능 확인용 설정이다.
- LCD 전원 투입 초기화 시간, 모터 속도, UART baud rate는 실물 적용 전에 설정해야 한다.
- ADC·DAC·SRAM·서보의 장비 제공 소스는 보존했다. 새 계층의 장치 모델과 검증은 후속 작업이다.



실습별 VCD 시간 구간 / 1

기준: 제공 테스트벤치 실행, 단위 ns. VCD 뷰어가 ps이면 1,000배한다.

구간 이름	시작(ns)	끝(ns)
01-logic-gates	1300	1340
02-not	1340	1360
03-full-adder	1360	1440
04-full-subtractor	1440	1520
05-encoder	1520	4080
06-decoder	4080	4160
07-mux	4160	4800
08-demux	4800	4960
09-compare	4960	7520
13-adder4	7520	10080
14-subtractor4	10080	12640



실습별 VCD 시간 구간 / 2

기준: 제공 테스트벤치 실행, 단위 ns. VCD 뷰어가 ps이면 1,000배한다.

구간 이름	시작(ns)	끝(ns)
15-multiplier4	12640	15200
00-first-counter	15200	15380
16-up-down-counter	15380	15540
17-divider	15540	16070
26-register	16070	16140
27-shift-register	16140	16290
28-piso	16290	16370
10-bcd-converter	16370	18960
11-seven-segment	18960	19120
12-digit-selection	19120	19200
32-lcd	19200	19680



실습별 VCD 시간 구간 / 3

기준: 제공 테스트벤치 실행, 단위 ns. VCD 뷰어가 ps이면 1,000배한다.

구간 이름	시작 (ns)	끝 (ns)
18-stepper	19680	19790
19-tone	19790	20460
20-piano	20460	21461
21-watch	21461	57490
22-pwm	57490	60530
23-rgb	60530	60620
24-rgb-pwm	60620	60970
29-moore	60970	61070
30-mealy	61070	61111
31-state-application	61111	61190
33-uart	61190	291620



half_adder.v / 코드

src/combinational/half_adder.v

L1-12

```
1  `timescale 1ns/1ps
2  `default_nettype none
3
4  // 반가산기: 두 입력을 더해 합과 자리올림을 만든다.
5  module half_adder (
6      input wire a,
7      input wire b,
8      output wire sum,
9      output wire carry
10 );
11     assign sum = a ^ b;
12     assign carry = a & b;
```



half_adder.v / 코드

src/combinational/half_adder.v

L13-15

```
13  endmodule  
14  
15  `default_nettype wire
```



lab_display.v / 코드

src/display/lab_display.v

L1-12

```
1  `timescale 1ns/1ps
2  `default_nettype none
3
4  // 표시 회로: 십진수 자리 선택과 LCD 명령 순서를 관찰한다.
5  module lab_display (
6      input wire clk,
7      input wire rst,
8      input wire enable,
9      input wire [7:0] value,
10     output wire [7:0] segments,
11     output wire [3:0] digit_select,
12     output wire [7:0] lcd_data,
```



lab_display.v / 코드

src/display/lab_display.v

L13-24

```
13     output wire lcd_e,  
14     output wire lcd_rs  
15 );  
16     reg [1:0] digit;  
17     reg [3:0] selected_value;  
18     wire [7:0] ones_full = value % 8'd10;  
19     wire [7:0] tens_full = (value / 8'd10) % 8'd10;  
20     wire [7:0] hundreds_full = value / 8'd100;  
21     wire [3:0] ones = ones_full[3:0];  
22     wire [3:0] tens = tens_full[3:0];  
23     wire [3:0] hundreds = hundreds_full[3:0];  
24     wire [6:0] bcd_binary;
```



lab_display.v / 코드

src/display/lab_display.v

L25-29

```
25     wire bcd_valid;
26     wire [7:0] manual_digits, manual_segments;
27
28     // 각 자리는 0~9이므로 하위 4비트만 사용한다.
29
```



lab_display.v / 코드

src/display/lab_display.v

L30-41

```
30     always @(posedge clk) begin
31         if (rst) digit <= 2'd0;
32         else if (enable) digit <= digit + 1'b1;
33     end
34     always @* begin
35         case (digit)
36             0: selected_value = ones;
37             1: selected_value = tens;
38             2: selected_value = hundreds;
39             default: selected_value = 4'd0;
40         endcase
41     end
```



lab_display.v / 코드

src/display/lab_display.v

L42-53

```
42     assign digit_select = 4'b0001 << digit;
43     hex7seg u_decoder (
44         .value(selected_value),
45         .segments(segments)
46     );
47
48     lcd_sequence u_lcd (
49         .clk(clk),
50         .rst(rst),
51         .tick(enable),
52         .data(lcd_data),
53         .e(lcd_e),
```



lab_display.v / 코드

src/display/lab_display.v

L54-63

```
54         .rs(lcd_rs)
55     );
56
57     bcd_to_binary u_bcd (
58         .tens(value[7:4]),
59         .ones(value[3:0]),
60         .binary(bcd_binary),
61         .valid(bcd_valid)
62     );
63
```



lab_display.v / 코드

src/display/lab_display.v

L64-73

```
64     segment_select8 u_manual (
65         .position(value[6:4]),
66         .digit(value[3:0]),
67         .digit_select(manual_digits),
68         .segments(manual_segments)
69     );
70
71 endmodule
72
73 `default_nettype wire
```



moore101.v / 코드

src/fsm/moore101.v

L1-12

```
1  `timescale 1ns/1ps
2  `default_nettype none
3
4  // Moore 검출기: 101 입력을 받은 뒤 상태에 따라 match가 변한다.
5  module moore101 (
6      input wire clk,
7      input wire rst,
8      input wire enable,
9      input wire bit_in,
10     output wire match
11 );
12     localparam IDLE = 2'd0, ONE = 2'd1, TEN = 2'd2, MATCH = 2'd3;
```



moore101.v / 코드

src/fsm/moore101.v

L13-24

```
13     reg [1:0] state, next_state;
14     always @* begin
15         case (state)
16             IDLE: next_state = bit_in ? ONE : IDLE;
17             ONE: next_state = bit_in ? ONE : TEN;
18             TEN: next_state = bit_in ? MATCH : IDLE;
19             MATCH: next_state = bit_in ? ONE : TEN;
20             default: next_state = IDLE;
21         endcase
22     end
23     always @(posedge clk) begin
24         if (rst) state <= IDLE;
```



moore101.v / 코드

src/fsm/moore101.v

L25-30

```
25         else if (enable) state <= next_state;
26     end
27     assign match = (state == MATCH);
28 endmodule
29
30 `default_nettype wire
```



mealy101.v / 코드

src/fsm/mealy101.v

L1-12

```
1  `timescale 1ns/1ps
2  `default_nettype none
3
4  // Mealy 검출기: 상태와 현재 입력으로 101을 검출한다.
5  module mealy101 (
6      input wire clk,
7      input wire rst,
8      input wire enable,
9      input wire bit_in,
10     output wire match
11 );
12     localparam IDLE = 2'd0, ONE = 2'd1, TEN = 2'd2;
```



mealy101.v / 코드

src/fsm/mealy101.v

L13-24

```
13     reg [1:0] state;
14     always @(posedge clk) begin
15         if (rst) state <= IDLE;
16         else if (enable) begin
17             case (state)
18                 IDLE: state <= bit_in ? ONE : IDLE;
19                 ONE: state <= bit_in ? ONE : TEN;
20                 TEN: state <= bit_in ? ONE : IDLE;
21                 default: state <= IDLE;
22             endcase
23         end
24     end
```



mealy101.v / 코드

src/fsm/mealy101.v

L25-28

```
25      assign match = enable && (state == TEN) && bit_in;  
26  endmodule  
27  
28  `default_nettype wire
```



lab_fsm.v / 코드

src/fsm/lab_fsm.v

L1-12

```
1  `timescale 1ns/1ps
2  `default_nettype none
3
4  // 상태 머신 묶음: Moore, Mealy, 순환 신호등을 비교한다.
5  module lab_fsm (
6      input wire clk,
7      input wire rst,
8      input wire enable,
9      input wire bit_in,
10     output wire [4:0] result
11 );
12     reg [2:0] traffic;
```



lab_fsm.v / 코드

src/fsm/lab_fsm.v

L13-24

```
13    wire moore_match, mealy_match;
14    wire [1:0] cycle_state, cycle_value, toggle_value, mode;
15    wire toggle_state;
16    wire [2:0] mode_rgb;
17    wire [3:0] mode_motor;
18
19    moore101 u_moore (
20        .clk(clk),
21        .rst(rst),
22        .enable(enable),
23        .bit_in(bit_in),
24        .match(moore_match)
```



lab_fsm.v / 코드

src/fsm/lab_fsm.v

L25-34

```
25     );  
26  
27     mealy101 u_mealy (  
28         .clk(clk),  
29         .rst(rst),  
30         .enable(enable),  
31         .bit_in(bit_in),  
32         .match(mealy_match)  
33     );  
34
```



lab_fsm.v / 코드

src/fsm/lab_fsm.v

L35-46

```
35     always @(posedge clk) begin
36         if (rst) traffic <= 3'b100;
37         else if (enable) traffic <= {traffic[1:0], traffic[2]};
38     end
39     assign result = {traffic, mealy_match, moore_match};
40
41     moore_cycle u_cycle (
42         .clk(clk),
43         .rst(rst),
44         .enable(enable),
45         .advance(bit_in),
46         .state(cycle_state),
```



lab_fsm.v / 코드

src/fsm/lab_fsm.v

L47-58

```
47         .value(cycle_value)
48     );
49
50     mealy_toggle u_toggle (
51         .clk(clk),
52         .rst(rst),
53         .enable(enable),
54         .bit_in(bit_in),
55         .state(toggle_state),
56         .value(toggle_value)
57     );
58
```



lab_fsm.v / 코드

src/fsm/lab_fsm.v

L59-68

```
59     mode_controller u_mode (  
60         .clk(clk),  
61         .rst(rst),  
62         .enable(enable),  
63         .next_mode(enable && bit_in),  
64         .mode(mode),  
65         .rgb(mode_rgb),  
66         .motor(mode_motor)  
67     );  
68
```



lab_fsm.v / 코드

src/fsm/lab_fsm.v

L69-71

```
69  endmodule  
70  
71  `default_nettype wire
```



uart_transmitter.v / 코드

src/interface/uart_transmitter.v

L1-12

```
1  `timescale 1ns/1ps
2  `default_nettype none
3
4  // UART 송신: 시작 비트 + 데이터 8비트(LSB 먼저) + 정지 비트.
5  module uart_transmitter #(parameter CLKS_PER_BIT = 8) (
6      input wire clk,
7      input wire rst,
8      input wire start,
9      input wire [7:0] data,
10     output wire tx,
11     output reg busy
12 );
```



uart_transmitter.v / 코드

src/interface/uart_transmitter.v

L13-24

```
13     reg [9:0] frame;
14     integer timer;
15     reg [3:0] bit_index;
16     assign tx = busy ? frame[bit_index] : 1'b1;
17     always @(posedge clk) begin
18         if (rst) begin
19             frame <= 10'h3ff;
20             timer <= 0;
21             bit_index <= 0;
22             busy <= 0;
23         end else if (!busy) begin
24             if (start) begin
```



uart_transmitter.v / 코드

src/interface/uart_transmitter.v

L25-34

```
25         frame <= {1'b1, data, 1'b0};
26         timer <= 0;
27         bit_index <= 0;
28         busy <= 1;
29     end
30 end else if (timer == CLKS_PER_BIT - 1) begin
31     timer <= 0;
32     if (bit_index == 9) busy <= 0;
33     else bit_index <= bit_index + 1'b1;
34 end else timer <= timer + 1;
```



uart_transmitter.v / 코드

src/interface/uart_transmitter.v

L35-38

```
35         end
36     endmodule
37
38     `default_nettype wire
```



uart_receiver.v / 코드

src/interface/uart_receiver.v

L1-12

```
1  `timescale 1ns/1ps
2  `default_nettype none
3
4  // UART 수신: 동일 클록의 내부 루프백용 교육 모델.
5  module uart_receiver #(parameter CLKS_PER_BIT = 8) (
6      input wire clk,
7      input wire rst,
8      input wire rx,
9      output reg [7:0] data,
10     output reg valid
11 );
12     localparam [1:0] IDLE = 2'd0;
```



uart_receiver.v / 코드

src/interface/uart_receiver.v

L13-24

```
13    localparam [1:0] START = 2'd1;
14    localparam [1:0] DATA = 2'd2;
15    localparam [1:0] STOP = 2'd3;
16    reg [1:0] state;
17    reg [7:0] shift;
18    reg [2:0] bit_index;
19    integer timer;
20    always @(posedge clk) begin
21        if (rst) begin
22            state <= IDLE;
23            timer <= 0;
24            bit_index <= 0;
```



uart_receiver.v / 코드

src/interface/uart_receiver.v

L25-34

```
25         shift <= 0;
26         data <= 0;
27         valid <= 0;
28     end else begin
29         valid <= 0;
30         case (state)
31             IDLE: if (!rx) begin
32                 state <= START;
33                 timer <= 0;
34             end
```



uart_receiver.v / 코드

src/interface/uart_receiver.v

L35-46

```
35         START: if (timer == CLKS_PER_BIT / 2 - 1) begin
36             timer <= 0;
37             if (!rx) begin
38                 state <= DATA;
39                 bit_index <= 0;
40             end
41             else state <= IDLE;
42         end else timer <= timer + 1;
43         DATA: if (timer == CLKS_PER_BIT - 1) begin
44             timer <= 0;
45             shift[bit_index] <= rx;
46             if (bit_index == 7) state <= STOP;
```



uart_receiver.v / 코드

src/interface/uart_receiver.v

L47-58

```
47         else bit_index <= bit_index + 1'b1;
48     end else timer <= timer + 1;
49     STOP: if (timer == CLKS_PER_BIT - 1) begin
50         state <= IDLE;
51         timer <= 0;
52         if (rx) begin
53             data <= shift;
54             valid <= 1;
55         end
56     end else timer <= timer + 1;
57     default: state <= IDLE;
58 endcase
```



uart_receiver.v / 코드

src/interface/uart_receiver.v

L59-63

```
59         end
60     end
61 endmodule
62
63 `default_nettype wire
```



shift_register8.v / 코드

src/sequential/shift_register8.v

L1-12

```
1 `timescale 1ns/1ps
2 `default_nettype none
3
4 // 8비트 레지스터: 병렬 적재 후 MSB부터 직렬 출력한다.
5 module shift_register8 (
6     input wire clk,
7     input wire rst,
8     input wire enable,
9     input wire load,
10    input wire serial_in,
11    input wire [7:0] data,
12    output reg [7:0] value
```



shift_register8.v / 코드

src/sequential/shift_register8.v

L13-21

```
13 );  
14     always @(posedge clk) begin  
15         if (rst) value <= 8'd0;  
16         else if (load) value <= data;  
17         else if (enable) value <= {value[6:0], serial_in};  
18     end  
19 endmodule  
20  
21 `default_nettype wire
```



lab_sequential.v / 코드

src/sequential/lab_sequential.v

L1-12

```
1  `timescale 1ns/1ps
2  `default_nettype none
3
4  // 순차회로 묶음: 카운터와 병렬 입력/직렬 출력 레지스터.
5  module lab_sequential (
6      input wire clk,
7      input wire rst,
8      input wire enable,
9      input wire load,
10     input wire down,
11     input wire serial_in,
12     input wire [7:0] data,
```



lab_sequential.v / 코드

src/sequential/lab_sequential.v

L13-24

```
13     output wire [15:0] result
14 );
15     wire [3:0] count;
16     wire [7:0] shift_value;
17     wire direct_clock, divided2, divided10, divided50;
18     wire [3:0] stored4, loaded4, shift4, parallel4;
19     wire serial4;
20
21     counter4 u_counter (
22         .clk(clk),
23         .rst(rst),
24         .enable(enable),
```



lab_sequential.v / 코드

src/sequential/lab_sequential.v

L25-28

```
25         .down(down),  
26         .count(count)  
27     );  
28
```



lab_sequential.v / 코드

src/sequential/lab_sequential.v

L29-40

```
29     shift_register8 u_shift (  
30         .clk(clk),  
31         .rst(rst),  
32         .enable(enable),  
33         .load(load),  
34         .serial_in(serial_in),  
35         .data(data),  
36         .value(shift_value)  
37     );  
38  
39     // count[3] is also an observable divide-by-16 waveform when counting up.  
40     assign result = {3'b000, shift_value[7], shift_value, count};
```



lab_sequential.v / 코드

src/sequential/lab_sequential.v

L41-52

```
41
42     clock_divider u_divider (
43         .clk(clk),
44         .rst(rst),
45         .enable(enable),
46         .direct_clock(direct_clock),
47         .div2(divided2),
48         .div10(divided10),
49         .div50(divided50)
50     );
51
52     register4 u_register4 (
```



lab_sequential.v / 코드

src/sequential/lab_sequential.v

L53-61

```
53         .clk(clk),  
54         .rst(rst),  
55         .save(load),  
56         .show(down),  
57         .data(data[3:0]),  
58         .stored(stored4),  
59         .value(loaded4)  
60     );  
61
```



lab_sequential.v / 코드

src/sequential/lab_sequential.v

L62-73

```
62     shift_register4 u_shift4 (  
63         .clk(clk),  
64         .rst(rst),  
65         .enable(enable),  
66         .serial_in(serial_in),  
67         .value(shift4)  
68     );  
69  
70     piso4 u_piso4 (  
71         .clk(clk),  
72         .rst(rst),  
73         .load(load),
```



lab_sequential.v / 코드

src/sequential/lab_sequential.v

L74-82

```
74         .enable(enable),  
75         .data(data[3:0]),  
76         .value(parallel4),  
77         .serial_out(serial4)  
78     );  
79  
80 endmodule  
81  
82 `default_nettype wire
```



top_legacy.v / 코드

src/top_legacy.v

L1-12

```
1  `timescale 1ns/1ps
2  `default_nettype none
3
4  // 교재용 통합 탑: 6개 주제의 모듈을 연결한다.
5  // 시뮬레이션/코드 설명용이며 보드 핀맵이나 장치 타이밍은 포함하지 않는다.
6  module top_legacy (
7      input wire clk,
8      input wire rst,
9      input wire enable,
10     input wire load,
11     input wire serial_in,
12     input wire start,
```



top_legacy.v / 코드

src/top_legacy.v

L13-24

```
13     input wire [7:0] a,  
14     input wire [7:0] b,  
15     input wire [2:0] select,  
16     output wire [31:0] comb_result,  
17     output wire [15:0] seq_result,  
18     output wire [7:0] segments,  
19     output wire [3:0] digit_select,  
20     output wire [7:0] lcd_data,  
21     output wire lcd_e,  
22     output wire lcd_rs,  
23     output wire [7:0] wave_result,  
24     output wire [4:0] fsm_result,
```



top_legacy.v / 코드

src/top_legacy.v

L25-34

```
25     output wire uart_tx,  
26     output wire uart_valid,  
27     output wire uart_busy,  
28     output wire [7:0] uart_data  
29 );  
30     lab_combinational u_comb (  
31         .a(a),  
32         .b(b),  
33         .select(select),  
34         .result(comb_result)
```



top_legacy.v / 코드

src/top_legacy.v

L35-46

```
35     );  
36  
37     lab_sequential u_seq (  
38         .clk(clk),  
39         .rst(rst),  
40         .enable(enable),  
41         .load(load),  
42         .down(select[0]),  
43         .serial_in(serial_in),  
44         .data(a),  
45         .result(seq_result)  
46     );
```



top_legacy.v / 코드

src/top_legacy.v

L47-58

```
47
48     lab_display u_display (
49         .clk(clk),
50         .rst(rst),
51         .enable(enable),
52         .value(a),
53         .segments(segments),
54         .digit_select(digit_select),
55         .lcd_data(lcd_data),
56         .lcd_e(lcd_e),
57         .lcd_rs(lcd_rs)
58     );
```



top_legacy.v / 코드

src/top_legacy.v

L59-59

59



top_legacy.v / 코드

src/top_legacy.v

L60-71

```
60     lab_waveform u_waveform (  
61         .clk(clk),  
62         .rst(rst),  
63         .enable(enable),  
64         .duty(a[3:0]),  
65         .keys(a),  
66         .note(select),  
67         .result(wave_result)  
68     );  
69  
70     lab_fsm u_fsm (  
71         .clk(clk),
```



top_legacy.v / 코드

src/top_legacy.v

L72-83

```
72         .rst(rst),
73         .enable(enable),
74         .bit_in(serial_in),
75         .result(fsm_result)
76     );
77
78     lab_interface u_interface (
79         .clk(clk),
80         .rst(rst),
81         .start(start),
82         .data(a),
83         .tx(uart_tx),
```



top_legacy.v / 코드

src/top_legacy.v

L84-90

```
84         .received(uart_data),  
85         .valid(uart_valid),  
86         .busy(uart_busy)  
87     );  
88 endmodule  
89  
90 `default_nettype wire
```



테스트벤치 읽는 순서

- 클록·reset과 DUT 포트 연결을 먼저 확인한다.
- 검사 task에서 기대값과 실제값의 비교를 읽는다.
- 실습별 입력 생성과 경계값 검사를 확인한다.
- 마지막 UART 전수 검사와 종료 조건까지 하나의 테스트벤치에서 실행한다.



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L1-12

```
1  `timescale 1ns/1ps
2  // 하나의 테스트벤치가 6개 주제를 순서대로 검사한다.
3  // 입력은 주로 하강 에지에서 바꾸고 상승 에지 후 결과를 확인한다.
4  module tb_legacy;
5      reg clk = 0;
6      always #5 clk = ~clk;
7      reg rst = 1, enable = 0, load = 0, serial_in = 0, start = 0;
8      reg [7:0] a = 0, b = 0;
9      reg [2:0] select = 0;
10     wire [31:0] comb_result;
11     wire [15:0] seq_result;
12     wire [7:0] segments, lcd_data, wave_result, uart_data;
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L13-24

```
13    wire [3:0] digit_select;
14    wire [4:0] fsm_result;
15    wire lcd_e, lcd_rs, uart_tx, uart_valid, uart_busy;
16    reg [2:0] group_id = 0;
17    // 파형의 구간 표시: 1~6 = 아래 테스트 순서
18    integer checks = 0;
19    integer i, j, high_count;
20    reg [4:0] expected_difference;
21
22    top_legacy dut (
23        .clk(clk),
24        .rst(rst),
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L25-34

```
25      .enable(enable),  
26      .load(load),  
27      .serial_in(serial_in),  
28      .start(start),  
29      .a(a),  
30      .b(b),  
31      .select(select),  
32      .comb_result(comb_result),  
33      .seq_result(seq_result),  
34      .segments(segments),
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L35-46

```
35         .digit_select(digit_select),  
36         .lcd_data(lcd_data),  
37         .lcd_e(lcd_e),  
38         .lcd_rs(lcd_rs),  
39         .wave_result(wave_result),  
40         .fsm_result(fsm_result),  
41         .uart_tx(uart_tx),  
42         .uart_valid(uart_valid),  
43         .uart_busy(uart_busy),  
44         .uart_data(uart_data)  
45     );  
46
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L47-58

```
47     task check(input bit condition, input string description);
48         checks++;
49         if (!condition) $fatal(1, "FAIL group %0d: %s at %0t", group_id, description, $time);
50     endtask
51
52     task cycle;
53         @(posedge clk);
54         #1;
55         @(negedge clk);
56     endtask
57
58     task reset_all;
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L59-68

```
59      @(negedge clk);
60      rst = 1;
61      enable = 0;
62      load = 0;
63      start = 0;
64      serial_in = 0;
65      repeat (2) cycle();
66      rst = 0;
67  endtask
68
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L69-80

```
69     task test_comb;
70         group_id = 3'd1;
71         $display("GROUP combinational");
72         for (i = 0; i < 16; i++) begin
73             for (j = 0; j < 16; j++) begin
74                 a = i[7:0];
75                 b = j[7:0];
76                 #1;
77                 expected_difference = i[4:0] - j[4:0];
78                 check(comb_result[4:0] === (i[4:0] + j[4:0]), "4-bit sum");
79                 check(comb_result[9:5] === expected_difference, "subtraction");
80                 check(comb_result[17:10] === (i[7:0] * j[7:0]), "multiplication");
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L81-92

```
81             check(comb_result[27] === (i > j), "comparison greater");
82             check(comb_result[28] === (i == j), "comparison equal");
83         end
84     end
85     a = 8'ha5;
86     for (i = 0; i < 8; i++) begin
87         select = i[2:0];
88         #1;
89         check(comb_result[25:18] === (8'b1 << i), "decoder");
90         check(comb_result[26] === a[i], "mux");
91     end
92 endtask
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L93-93

93



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L94-105

```
94     task test_seq;
95         group_id = 3'd2;
96         $display("GROUP sequential");
97         reset_all();
98         select = 0;
99         enable = 1;
100        repeat (16) cycle();
101        check(seq_result[3:0] === 4'd0, "counter overflow");
102        select = 1;
103        cycle();
104        check(seq_result[3:0] === 4'd15, "counter underflow");
105        enable = 0;
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L106-117

```
106      cycle();
107      check(seq_result[3:0] === 4'd15, "counter hold");
108      a = 8'ha5;
109      load = 1;
110      cycle();
111      load = 0;
112      check(seq_result[11:4] === 8'ha5, "parallel load");
113      enable = 1;
114      serial_in = 0;
115      for (i = 7; i >= 0; i--) begin
116          check(seq_result[12] === a[i], "PISO MSB-first");
117          cycle();
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L118-121

```
118         end
119         check(seq_result[11:4] === 8'd0, "shift complete");
120     endtask
121
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L122-133

```
122     task test_display;
123         group_id = 3'd3;
124         $display("GROUP display");
125         reset_all();
126         a = 123;
127         #1;
128         check(segments === 8'hf2, "BCD ones = 3");
129         check(digit_select === 4'b0001, "first digit");
130         enable = 1;
131         cycle();
132         check(segments === 8'hda, "BCD tens = 2");
133         cycle();
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L134-145

```
134     check(segments === 8'h60, "BCD hundreds = 1");
135     cycle();
136     check(segments === 8'hfc, "leading zero");
137     reset_all();
138     enable = 1;
139     cycle();
140     check(lcd_data === 8'h38 && !lcd_e && !lcd_rs, "LCD setup");
141     cycle();
142     check(lcd_e === 1'b1, "LCD enable high");
143     cycle();
144     check(lcd_e === 1'b0, "LCD enable low");
145     repeat (9) cycle();
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L146-149

```
146         cycle();  
147         check(lcd_data === 8'h48 && lcd_rs, "LCD character H");  
148     endtask  
149
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L150-161

```
150     task test_waveform;
151         group_id = 3'd4;
152         $display("GROUP waveform");
153         reset_all();
154         a = 4;
155         enable = 1;
156         high_count = 0;
157         repeat (16) begin
158             cycle();
159             if (wave_result[0]) high_count++;
160         end
161         check(high_count == 4, "PWM 4/16 duty");
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L162-173

```
162     check(wave_result[7:4] === 4'b1001, "motor full cycle");
163     a = 0;
164     #1;
165     check(wave_result[0] === 1'b0, "PWM zero duty");
166     reset_all();
167     select = 0;
168     enable = 1;
169     repeat (15) cycle();
170     check(wave_result[3] === 1'b0, "tone before half-period");
171     cycle();
172     check(wave_result[3] === 1'b1, "tone half-period");
173     endtask
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L174-174

174



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L175-186

```
175     task test_fsm;
176         group_id = 3'd5;
177         $display("GROUP fsm");
178         reset_all();
179         enable = 1;
180         serial_in = 1;
181         cycle();
182         serial_in = 0;
183         cycle();
184         serial_in = 1;
185         #1;
186         check(fsm_result[1] === 1'b1, "Mealy before edge");
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L187-198

```
187         check(fsm_result[0] === 1'b0, "Moore before edge");
188         cycle();
189         check(fsm_result[0] === 1'b1, "Moore after edge");
190         serial_in = 0;
191         cycle();
192         serial_in = 1;
193         cycle();
194         check(fsm_result[0] === 1'b1, "overlapping 10101");
195     endtask
196
197     task test_uart;
198         group_id = 3'd6;
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L199-208

```
199      $display("GROUP interface UART 8N1");
200      reset_all();
201      for (i = 0; i < 256; i++) begin
202          a = i[7:0];
203          start = 1;
204          cycle();
205          start = 0;
206          check(uart_busy === 1'b1, "UART busy while sending");
207          wait (uart_valid);
208          #1;
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L209-220

```
209         check(uart_data === a, "UART byte loopback");
210         repeat (12) cycle();
211         check(uart_busy === 1'b0 && uart_tx === 1'b1, "UART idle after stop bit");
212     end
213 endtask
214
215
216 // 촬영용 구간 목록. 마지막 행은 finish_capture에서 닫는다.
217 integer capture_file;
218 time capture_start;
219 string capture_name = "";
220 reg [7:0] experiment_id = 0;
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L221-231

```
221
222     task begin_capture(input integer number, input string name);
223         if (capture_name != "")
224             $fdisplay(capture_file, "%s,%0d,%0d", capture_name,
225                     capture_start, $time);
226         capture_name = name;
227         capture_start = $time;
228         experiment_id = number[7:0];
229         $display("CAPTURE %0d %s at %0t", experiment_id, name, $time);
230     endtask
231
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L232-243

```
232     function automatic [7:0] digit_pattern(input integer digit);
233         case (digit)
234             0: digit_pattern = 8'hfc;
235             1: digit_pattern = 8'h60;
236             2: digit_pattern = 8'hda;
237             3: digit_pattern = 8'hf2;
238             4: digit_pattern = 8'h66;
239             5: digit_pattern = 8'hb6;
240             6: digit_pattern = 8'hbe;
241             7: digit_pattern = 8'he0;
242             8: digit_pattern = 8'hfe;
243             9: digit_pattern = 8'hf6;
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L244-255

```
244             10: digit_pattern = 8'hee;
245             11: digit_pattern = 8'h3e;
246             12: digit_pattern = 8'h9c;
247             13: digit_pattern = 8'h7a;
248             14: digit_pattern = 8'h9e;
249             default: digit_pattern = 8'h8e;
250         endcase
251     endfunction
252
253     task test_book_comb;
254         integer k;
255         reg [1:0] diff;
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L256-265

```
256     group_id = 3'd1;
257     reset_all();
258     begin_capture(1, "01-logic-gates");
259     for (i = 0; i < 4; i++) begin
260         a = {7'd0, i[1]};
261         b = {7'd0, i[0]};
262         #10;
263         check(dut.u_comb.gate_and == (a[0] & b[0]), "AND");
264         check(dut.u_comb.gate_or == (a[0] | b[0]), "OR");
265         check(dut.u_comb.gate_xor == (a[0] ^ b[0]), "XOR");
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L266-277

```
266     end
267     begin_capture(2, "02-not");
268     for (i = 0; i < 2; i++) begin
269         a = i[7:0]; #10;
270         check(dut.u_comb.gate_not === ~a[0], "NOT");
271     end
272     begin_capture(3, "03-full-adder");
273     for (i = 0; i < 8; i++) begin
274         // bit0의 1+1로 bit1에 carry_in=1을 공급한다.
275         a = {6'd0, i[2], i[0]};
276         b = {6'd0, i[1], i[0]};
277         #10;
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L278-289

```
278         check(dut.u_comb.u_bit1.sum === (i[2] ^ i[1] ^ i[0]), "FA sum");
279         check(dut.u_comb.u_bit1.carry_out === ((i[2] & i[1]) | ((i[2] ^ i[1]) & i[0])), "FA carry");
280     end
281     begin_capture(4, "04-full-subtractor");
282     for (i = 0; i < 8; i++) begin
283         a = {7'd0, i[2]}; b = {7'd0, i[1]};
284         select = {2'd0, i[0]};
285         diff = {1'b0, i[2]} - {1'b0, i[1]} - {1'b0, i[0]};
286         #10;
287         check({dut.u_comb.borrow, dut.u_comb.subtract_bit} === diff, "full subtractor");
288     end
289     begin_capture(5, "05-encoder");
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L290-299

```
290     for (i = 0; i < 256; i++) begin
291         a = i[7:0]; #10;
292         check(dut.u_comb.encoder_valid === (i != 0 && ((i & (i-1)) == 0)), "one-hot validity");
293         if (dut.u_comb.encoder_valid) begin
294             for (k = 0; k < 8; k++)
295                 if (a[7-k]) check(dut.u_comb.encoded === k[2:0], "MSB-first encoder");
296             end else check(dut.u_comb.encoded === 3'd0, "invalid encoder default");
297         end
298         begin_capture(6, "06-decoder");
299         for (i = 0; i < 8; i++) begin
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L300-311

```
300         select = i[2:0]; #10;
301         check(comb_result[25:18] === (8'b1 << i), "decoder all selections");
302     end
303     begin_capture(7, "07-mux");
304     for (i = 0; i < 16; i++) begin
305         a = i[7:0];
306         for (j = 0; j < 4; j++) begin
307             select = j[2:0]; #10;
308             check(dut.u_comb.mux4 === a[3-j], "4:1 MSB-first mux");
309         end
310     end
311     begin_capture(8, "08-demux");
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L312-323

```
312         for (i = 0; i < 2; i++) begin
313             b = i[7:0];
314             for (j = 0; j < 8; j++) begin
315                 select = j[2:0]; #10;
316                 check(dut.u_comb.demux8 === (b[0] ? (8'h80 >> j) : 8'd0), "1:8 demux");
317             end
318         end
319         begin_capture(9, "09-compare");
320         for (i = 0; i < 16; i++) begin
321             for (j = 0; j < 16; j++) begin
322                 a = i[7:0]; b = j[7:0]; #10;
323                 check(comb_result[28:27] === {i == j, i > j}, "comparison");
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L324-333

```
324         end
325     end
326     for (k = 13; k <= 15; k++) begin
327         case (k)
328             13: begin_capture(k, "13-adder4");
329             14: begin_capture(k, "14-subtractor4");
330             15: begin_capture(k, "15-multiplier4");
331             default: $fatal(1, "Invalid arithmetic capture");
332         endcase
333     for (i = 0; i < 16; i++) begin
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L334-345

```
334         for (j = 0; j < 16; j++) begin
335             a = i[7:0]; b = j[7:0]; #10;
336             check(comb_result[4:0] === i[4:0] + j[4:0], "sum");
337             check(comb_result[9:5] === i[4:0] - j[4:0], "difference");
338             check(comb_result[17:10] === i[7:0] * j[7:0], "product");
339         end
340     end
341 end
342 endtask
343
344 task test_book_seq;
345     reg [3:0] expected_shift;
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L346-357

```
346      group_id = 3'd2;
347      begin_capture(0, "00-first-counter");
348      reset_all(); select = 0; enable = 1;
349      for (i = 1; i <= 16; i++) begin
350          cycle(); check(seq_result[3:0] === i[3:0], "up count");
351      end
352      begin_capture(16, "16-up-down-counter");
353      select = 1;
354      for (i = 15; i >= 0; i--) begin
355          cycle(); check(seq_result[3:0] === i[3:0], "down count");
356      end
357      begin_capture(17, "17-divider");
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L358-367

```
358     reset_all(); enable = 1;
359     for (i = 1; i <= 50; i++) begin
360         cycle();
361         check(dut.u_seq.divided2 === ((i % 2) != 0), "divide by 2");
362         check(dut.u_seq.divided10 === (((i / 5) % 2) != 0), "divide by 10");
363         check(dut.u_seq.divided50 === (((i / 25) % 2) != 0), "divide by 50");
364         check(dut.u_seq.direct_clock === clk, "direct clock");
365     end
366     begin_capture(26, "26-register");
367     reset_all(); select = 0;
```

통합 테스트벤치 / 코드

sim/tb_legacy.sv

L368-379

```
368     a = 8'ha; load = 1; cycle(); load = 0;
369     a = 8'h3; cycle();
370     check(dut.u_seq.stored4 === 4'ha && dut.u_seq.loaded4 === 4'd0, "saved data held");
371     select = 1; cycle(); select = 0;
372     check(dut.u_seq.loaded4 === 4'ha, "show stored value");
373     load = 1; select = 1; cycle();
374     check(dut.u_seq.stored4 === 4'h3 && dut.u_seq.loaded4 === 4'ha, "simultaneous uses old stored value");
375     begin_capture(27, "27-shift-register");
376     reset_all(); expected_shift = 0; enable = 1;
377     for (i = 0; i < 12; i++) begin
378         serial_in = (i % 3 == 0);
379         expected_shift = {serial_in, expected_shift[3:1]};
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L380-391

```
380         cycle(); check(dut.u_seq.shift4 === expected_shift, "MSB input shift");
381     end
382     begin_capture(28, "28-piso");
383     reset_all(); a = 8'ha; load = 1; cycle(); load = 0;
384     check(dut.u_seq.parallel4 === 4'ha, "PISO load");
385     enable = 1;
386     for (i = 3; i >= 0; i--) begin
387         check(dut.u_seq.serial4 === a[i], "PISO output before shift");
388         cycle();
389     end
390     check(dut.u_seq.parallel4 === 4'd0, "PISO drained");
391 endtask
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L392-392

392



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L393-404

```
393     task test_book_display;
394         string message;
395         group_id = 3'd3;
396         begin_capture(10, "10-bcd-converter");
397         reset_all();
398         for (i = 0; i < 16; i++) begin
399             for (j = 0; j < 16; j++) begin
400                 a = {i[3:0], j[3:0]}; #10;
401                 check(dut.u_display.bcd_valid === (i < 10 && j < 10), "BCD valid digits");
402                 if (i < 10 && j < 10)
403                     check(dut.u_display.bcd_binary === (i[6:0] * 7'd10 + j[6:0]), "BCD conversion");
404                 else check(dut.u_display.bcd_binary === 7'd0, "invalid BCD output");
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L405-416

```
405         end
406     end
407     begin_capture(11, "11-seven-segment");
408     for (i = 0; i < 16; i++) begin
409         a = i[7:0]; #10;
410         check(dut.u_display.manual_segments === digit_pattern(i), "hex patterns");
411     end
412     begin_capture(12, "12-digit-selection");
413     for (i = 0; i < 8; i++) begin
414         a = {1'b0, i[2:0], 4'd8}; #10;
415         check(dut.u_display.manual_digits === (8'b1 << i), "8 digit selection");
416         check(dut.u_display.manual_segments === 8'hfe, "shared segment data");
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L417-426

```
417         end
418         begin_capture(32, "32-lcd");
419         reset_all(); enable = 1; message = "Hello World";
420         repeat (12) cycle();
421         for (i = 0; i < message.len(); i++) begin
422             cycle();
423             check(lcd_data === $unsigned(message.getc(i)) && lcd_rs && !lcd_e, "LCD data setup");
424             cycle(); check(lcd_e === 1'b1, "LCD E high");
425             cycle(); check(lcd_e === 1'b0, "LCD E low");
426         end
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L427-438

```
427     cycle(); check(!lcd_rs && lcd_data === 8'h80, "LCD restart address");
428     endtask
429
430     task test_book_waveform;
431         integer k, expected_digit, position;
432         group_id = 3'd4;
433         begin_capture(18, "18-stepper");
434         reset_all(); enable = 1;
435         for (i = 0; i < 8; i++) begin
436             case (i % 4)
437                 0: check(wave_result[7:4] === 4'b1001, "step 0");
438                 1: check(wave_result[7:4] === 4'b0011, "step 1");
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L439-450

```
439         2: check(wave_result[7:4] === 4'b0110, "step 2");
440         3: check(wave_result[7:4] === 4'b1100, "step 3");
441         default: $fatal(1, "Invalid step index");
442     endcase
443     cycle();
444 end
445 begin_capture(19, "19-tone");
446 reset_all(); select = 0; enable = 1;
447 for (i = 1; i <= 64; i++) begin
448     cycle(); check(wave_result[3] === (((i / 16) % 2) != 0), "tone period");
449 end
450 begin_capture(20, "20-piano");
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L451-460

```
451     for (k = 0; k < 8; k++) begin
452         reset_all(); a = 8'h80 >> k; enable = 1; #1;
453         check(dut.u_waveform.piano_active && dut.u_waveform.piano_note === k[2:0], "key selects note");
454         cycle(); // 새 키의 위상 초기화
455         repeat (4 + k) cycle();
456         check(dut.u_waveform.piano_tone === 1'b1, "piano half period");
457         a = 0; cycle();
458         check(!dut.u_waveform.piano_active && !dut.u_waveform.piano_tone, "piano silence");
459     end
460     a = 8'h81; #1;
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L461-472

```
461     check(dut.u_waveform.piano_note === 3'd0, "key priority");
462     begin_capture(21, "21-watch");
463     reset_all(); enable = 1;
464     for (i = 1; i <= 3600; i++) begin
465         cycle();
466         check(dut.u_waveform.seconds === $unsigned(6'(i % 60)), "watch seconds");
467         check(dut.u_waveform.minutes === $unsigned(6'((i / 60) % 60)), "watch minutes");
468         position = i % 4;
469         case (position)
470             0: expected_digit = (i % 60) % 10;
471             1: expected_digit = (i % 60) / 10;
472             2: expected_digit = ((i / 60) % 60) % 10;
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L473-484

```
473         default: expected_digit = ((i / 60) % 60) / 10;
474     endcase
475     check(dut.u_waveform.watch_digits === (4'b1 << position), "watch scan");
476     check(dut.u_waveform.watch_segments === digit_pattern(expected_digit), "watch displayed digit");
477 end
478 begin_capture(22, "22-pwm");
479 for (k = 0; k < 16; k++) begin
480     reset_all(); a = k[7:0]; enable = 1; high_count = 0;
481     repeat (16) begin
482         cycle(); if (wave_result[0]) high_count++;
483     end
484     check(high_count == k, "all PWM duty settings");
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L485-494

```
485     end
486     begin_capture(23, "23-rgb");
487     reset_all(); enable = 1;
488     for (i = 0; i < 6; i++) begin
489         check(dut.u_waveform.rgb_state === (3'b100 >> (i % 3)), "RGB sequence");
490         cycle();
491     end
492     begin_capture(24, "24-rgb-pwm");
493     reset_all(); enable = 1;
494     for (i = 1; i <= 32; i++) begin
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L495-506

```
495         cycle();
496         check(wave_result[1] === ((i % 16) < 4), "green PWM");
497         check(wave_result[2] === ((i % 16) < 12), "blue PWM");
498     end
499 endtask
500
501 task test_book_fsm;
502     group_id = 3'd5;
503     begin_capture(29, "29-moore");
504     reset_all(); enable = 1; serial_in = 1;
505     for (i = 1; i <= 6; i++) begin
506         cycle();
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L507-518

```
507         check(dut.u_fsm.cycle_state === $unsigned(2'(i % 3)), "Moore cycle state");
508         check(dut.u_fsm.cycle_value === dut.u_fsm.cycle_state, "Moore output");
509     end
510     serial_in = 0; cycle(); check(dut.u_fsm.cycle_value === 2'd0, "Moore hold");
511     begin_capture(30, "30-mealy");
512     reset_all(); enable = 1; serial_in = 1; #1;
513     check(dut.u_fsm.toggle_value === 2'b10, "Mealy initial input-dependent output");
514     cycle();
515     check(dut.u_fsm.toggle_state === 1'b1 && dut.u_fsm.toggle_value === 2'b01, "Mealy second state");
516     serial_in = 0; #1;
517     check(dut.u_fsm.toggle_value === 2'b00, "Mealy zero input");
518     begin_capture(31, "31-state-application");
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L519-528

```
519     reset_all(); enable = 1;
520     check(dut.u_fsm.mode === 2'd0 && dut.u_fsm.mode_rgb === 3'd0 && dut.u_fsm.mode_motor === 4'd0, "mode idle");
521     serial_in = 1; cycle(); serial_in = 0; cycle();
522     check(dut.u_fsm.mode === 2'd1 && dut.u_fsm.mode_rgb !== 3'd0 && dut.u_fsm.mode_motor === 4'd0, "mode RGB");
523     serial_in = 1; cycle(); serial_in = 0; cycle();
524     check(dut.u_fsm.mode === 2'd2 && dut.u_fsm.mode_rgb === 3'd0 && dut.u_fsm.mode_motor !== 4'd0, "mode motor");
525     serial_in = 1; cycle(); serial_in = 0;
526     check(dut.u_fsm.mode === 2'd0 && dut.u_fsm.mode_motor === 4'd0, "mode returns idle");
527 endtask
528
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L529-540

```
529     initial begin
530         $timeformat(-9, 0, " ns", 0);
531         capture_file = $fopen("capture_windows.csv", "w");
532         if (!capture_file) $fatal(1, "Cannot write capture_windows.csv");
533         $fdisplay(capture_file, "name,start_ns,end_ns");
534         begin_capture(255, "baseline");
535         reset_all();
536         test_comb();
537         test_seq();
538         test_display();
539         test_waveform();
540         test_fsm();
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L541-552

```
541      test_book_comb();
542      test_book_seq();
543      test_book_display();
544      test_book_waveform();
545      test_book_fsm();
546      begin_capture(33, "33-uart");
547      test_uart();
548      $fdisplay(capture_file, "%s,%0d,%0d", capture_name, capture_start, $time);
549      $fclose(capture_file);
550      $display("PASS: %0d checks, 6 groups", checks);
551      $finish;
552  end
```



통합 테스트벤치 / 코드

sim/tb_legacy.sv

L553-558

```
553
554     initial begin
555         #1000000;
556         $fatal(1, "Simulation timeout");
557     end
558 endmodule
```



출처와 보충 자료 / 구성

7. 출처와 보충 자료

교육 자료 출처

325

장비 제공 보충 소스

326



사용한 교육 자료

- 전자전기컴퓨터설계실습 II, 2022년 교안. 원자료의 절과 페이지는 TeX 원문에 기록했다.
- 신 장비 교안: Behavioral Modeling, Register and Counter, Finite State Machine, Character LCD Control의 개념 설명을 참고했다.
- Combo II 장비 제공 Example, FullDemo, Datasheet. 개인 실습 코드는 제외했다.
- 재구성 코드와 장비 제공 원본을 구분한다. 외부 공개·재배포 전 원자료의 이용 조건을 확인한다.



장비 제공 보충 소스

- vendor/examples/adc, dac, sram: 변환기와 외부 메모리
- vendor/examples/serial, usb232: 직렬 통신
- vendor/examples/servo_ctrl: 서보 제어
- vendor/full_demo: VHDL 통합 데모
- datasheets/: AD7908, AD7302, SRAM 등 부품 참고 자료

